University of Engineering & Management, Kolkata

Department of CST/CSIT/CSE (CYBER SECURITY)/ CSE (NETWORKS)

Paper Name: - Computer Networks Laboratory Manual

Paper Code: - PCCCS691

Session: - 2025-2026

Semester: - 6th

Faculty Name: -

NAME:

SECTION: ROLL NO:

STREAM: YEAR AND SEMESTER:

ENROLLMENT NO:

University of Engineering & Management, Kolkata

INDEX

SL NO	NAME OF EXPERIMENTS	PAGE NO	DATE OF EXPERIMENT	DATE OF SUBMISSION	SIGNATURE	REMARKS
1	Study of Network devices: Hub, Switches, Routers etc.					
2	(a) Construct Straight-Through and Cross-Over RJ-45 Ethernet cables. (b) Construct a peer-to-peer connection using cross-over ethernet cable.					
3	Study of network IP and set of Networking Commands.					
4	Study of ssh and FTP server configuration.					
5	(a) Implement a peer-to-peer connection in CISCO Packet Tracer. (b) Implement a LAN connection in CISCO Packet Tracer.					
6	Configure a Router on CISCO Packet Tracer using DHCP.					

SL NO	NAME OF EXPERIMENTS	PAGE NO	DATE OF EXPERIMENT	DATE OF SUBMISSION	SIGNATURE	REMARKS
7	(a) Implementation of Cyclic Redundancy Check (CRC) for Error Detection using C. (b) Implementation of Check-sum for Error detection using C.					
8	Implementation of client-server program with user input message with connection-less socket using C.					
9	Implementation of the client-server program with connection-oriented socket using C.					
10	Implementation of the multi-client and single Chat server program for the connection-oriented socket programming using C.					

EXPERIMENT NO: -1

Title: - Study of Network devices: Hub, Switches, Routers etc.

Objective: - To study of different network devices like Hub, Switches, Routers etc.

Theory: -

1. HUB: - Hub in networking plays a vital role in data transmission and broadcasting. A hub is a hardware device used at the physical layer to connect multiple devices in the network. Hubs are widely used to connect LANs. A hub has multiple ports. Unlike a switch, a hub cannot filter the data, i.e., it cannot identify the destination of the packet, so it broadcasts or sends the message to each port.

A hub is a multi-port repeater. A hub connects multiple wires coming from different branches, for example, the connector in star topology which connects different stations. Hubs cannot filter data, so data packets are sent to all connected devices. In other words, the collision domain of all hosts connected through hub remains one. Hub does not have any routing table to store the data of ports and map destination addresses., the routing table is used to send/broadcast information across all the ports.

How Does a Network Hub Work?

A hub is a multiport device, which has multiple ports in a device and shares the data to multiple ports altogether. A hub acts as a dumb switch that does not know, which data needs to be forwarded where so it broadcasts or sends the data to each port.

Suppose there are five ports in a hub A, B, C, D, and E. Consider A wants to send any data frame, or let's say A is acting as a sender, so the hub will forward the data transmitted by A to B, C, D, E. Now, at the same time B also wants to send the data then data received from A and B will collide and can cause data loss. In this situation, the data gets destroyed, and the hosts send a jam signal to all the hosts informing them about the collision, and each sender needs to wait for a certain amount of time.

Note: In the hub, data is sent to all ports but each port accepts only that data whose destination address matches their MAC address.

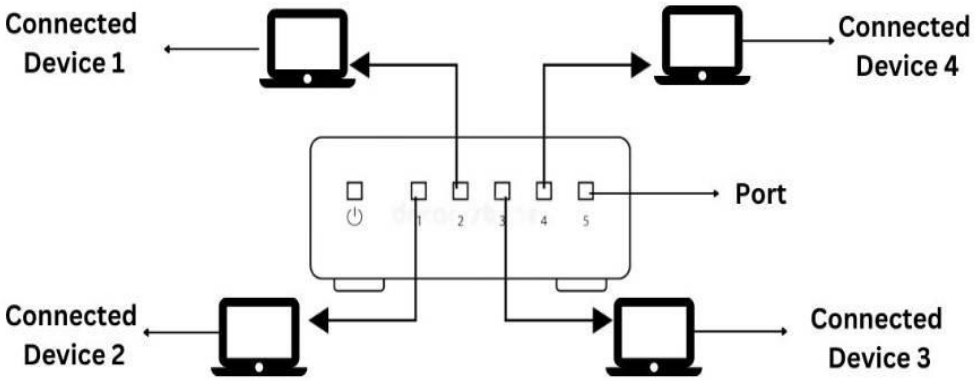
Types of Network Hubs

Networks hubs are classified into three types:

(a) Active Hub: They have a power supply for regenerating, and amplifying the signals. When a port sends weak signaled data, the hub regenerates the signal and strengthens it, then send it further to all other ports. Active hubs are expensive in costs as compared to passive hubs.

(b) Passive Hub: Passive hubs are simply used to connect signals from different network cables as they do not have any computerized element. They simply connect the wires of different devices in the star topology. Passive hubs do not do any processing or signal regeneration and that's why do not require electricity the most they can do is they can copy or repeat the signal. It can't clean the message, and it can't amplify or strengthen the signal.

(c) Intelligent Hub: Intelligent hubs as the name suggests are smarter than active and passive hubs. The intelligent hub comprises a special monitoring unit named a Management Information Base (MIB). This is software that helps in analysing and troubleshooting network problems. Intelligent hubs work similarly to active hubs but with some management features. Like it can monitor the traffic of the network and the configuration of a port.



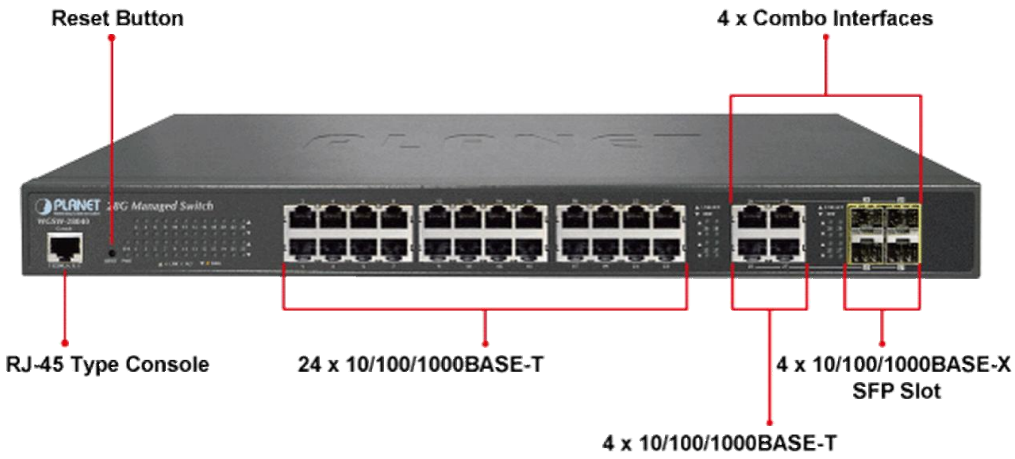
2. SWITCHES: - The Switch is a network device that is used to segment the networks into different subnetworks called subnets or LAN segments. It is responsible for filtering and forwarding the packets between LAN segments based on MAC address.

Switches have many ports, and when data arrives at any port, the destination address is examined first and some checks are also done and then it is processed to the devices. Different types of communication are supported here like unicast, multicast, and broadcast communication.

How Does a Network Switch Works?

When the source wants to send the data packet to the destination, the packet first enters the switch and the switch reads its header and finds the MAC address of the destination to identify the device then it sends the packet out through the appropriate ports that lead to the destination devices.

Switch establishes a temporary connection between the source and destination for communication and terminates the connection once the conversation is done. Also, it offers full bandwidth to network traffic going to and from a device simultaneously to reduce collision.



3. ROUTERS: - A router is a device that connects two or more packet-switched networks or subnetworks. It serves two primary functions: managing traffic between these networks by forwarding data packets to their intended IP addresses, and allowing multiple devices to use the same Internet connection.

There are several types of routers, but most routers pass data between LANs (local area networks) and WANs (wide area networks). A LAN is a group of connected devices restricted to a specific geographic area. A LAN usually requires a single router.

A WAN, by contrast, is a large network spread out over a vast geographic area. Large organizations and companies that operate in multiple locations across the country, for instance, will need separate LANs for each location, which then connect to the other LANs to form a WAN. Because a WAN is distributed over a large area, it often necessitates multiple routers and switches*.

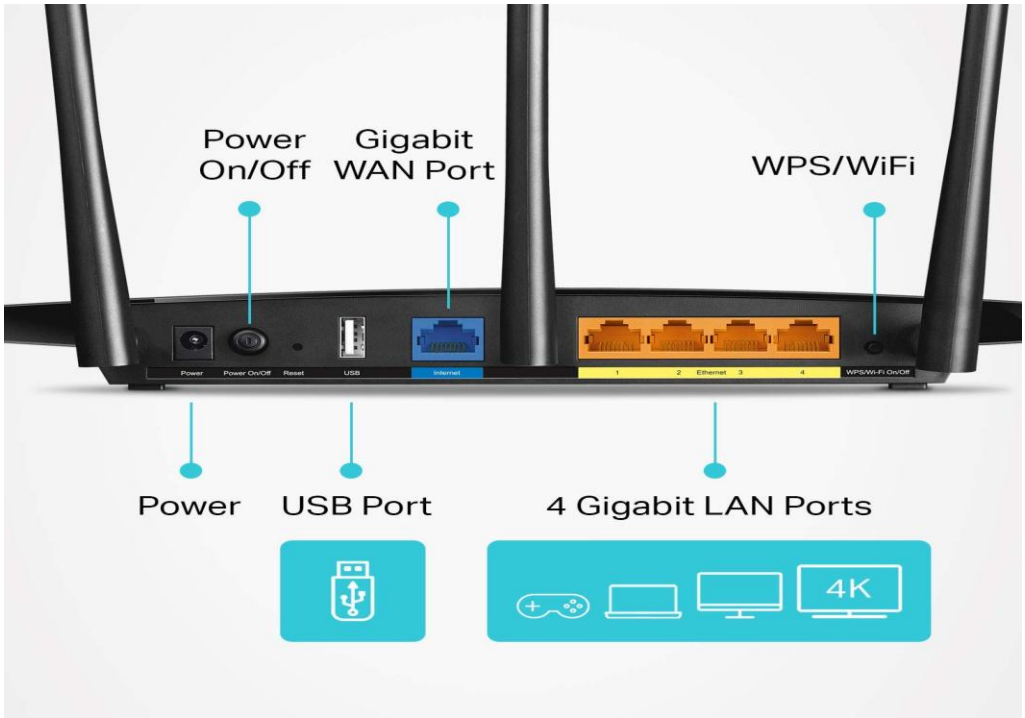
**A network switch forwards data packets between groups of devices in the same network, whereas a router forwards data between different networks.*

How does a router work?

Think of a router as an air traffic controller and data packets as aircraft headed to different airports (or networks). Just as each plane has a unique destination and follows a unique route, each packet needs to be guided to its destination as efficiently as possible. In the same way that an air traffic controller ensures that planes reach their destinations without getting lost or suffering a major disruption along the way, a router helps direct data packets to their destination IP address.

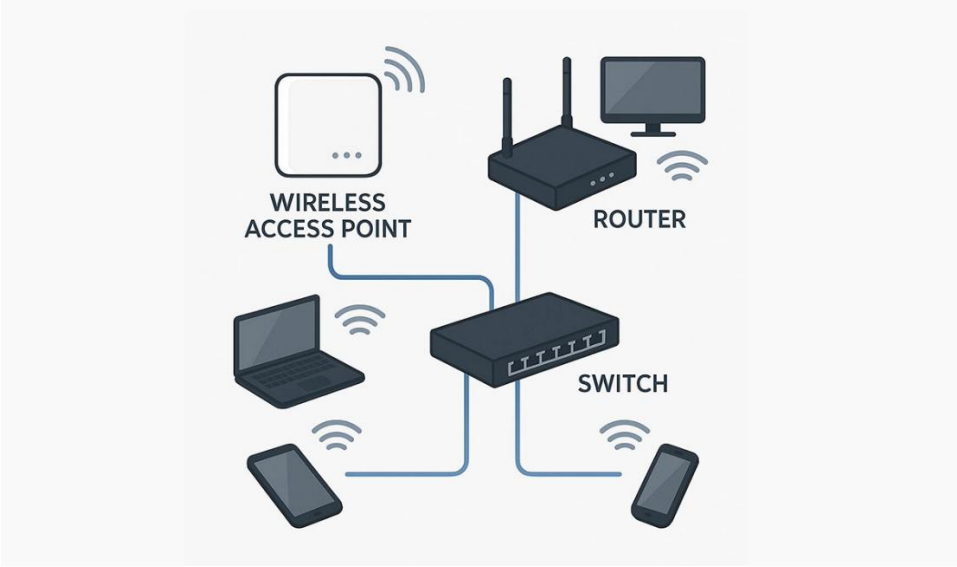
In order to direct packets effectively, a router uses an internal routing table — a list of paths to various network destinations. The router reads a packet's header to determine where it is going,

then consults the routing table to figure out the most efficient path to that destination. It then forwards the packet to the next network in the path.



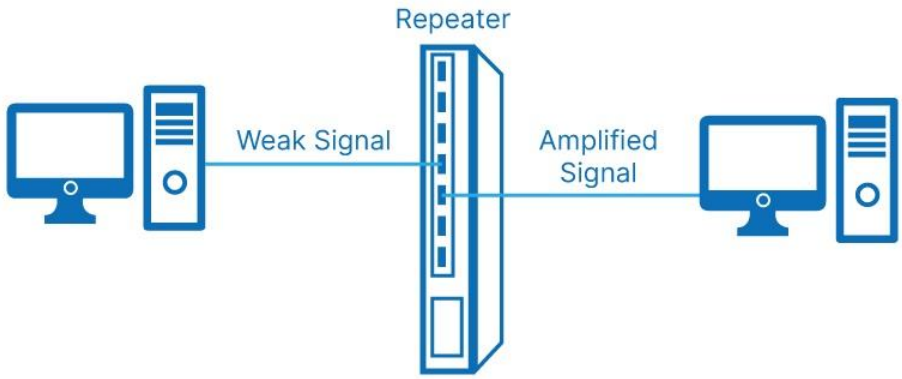
4. MODEM: - A modem and router are two of the most frequent components in a home network configuration. A router establishes a local area network (LAN), whereas a modem connects to an internet service provider (ISP). For a home network to work, both devices are necessary. Modem stands for Modulator/Demodulator. The modem is defined as a networking device that is used to connect devices connected in the network to the internet. The main function of a modem is to convert the analog signals that come from telephone wire into a digital form. In digital form, these converted signals are stored in the form of 0s and 1s. The modem can perform both the task of modulation and demodulation simultaneously. Modems are majorly used to transfer digital data in personal systems. The modem is also known as a signal translator as it translates one signal into another signal by modulating the digital signal into an analog signal for transmission and then demodulates receiving analog signals into digital signals.

5. WIRELESS ACCESS POINT (OR AP / WAP): - A Wireless Access Point (WAP or simply AP) enables wireless devices — laptops, smartphones, tablets — to connect to a wired network by creating a wireless local area network (WLAN). The AP connects to a wired network (often via a switch) and broadcasts Wi-Fi signals so that wireless devices can join the network and access shared resources or the Internet. Because APs provide flexibility (no wiring needed for every device), they are widely used in offices, educational institutions, public networks, or homes with multiple wireless devices.



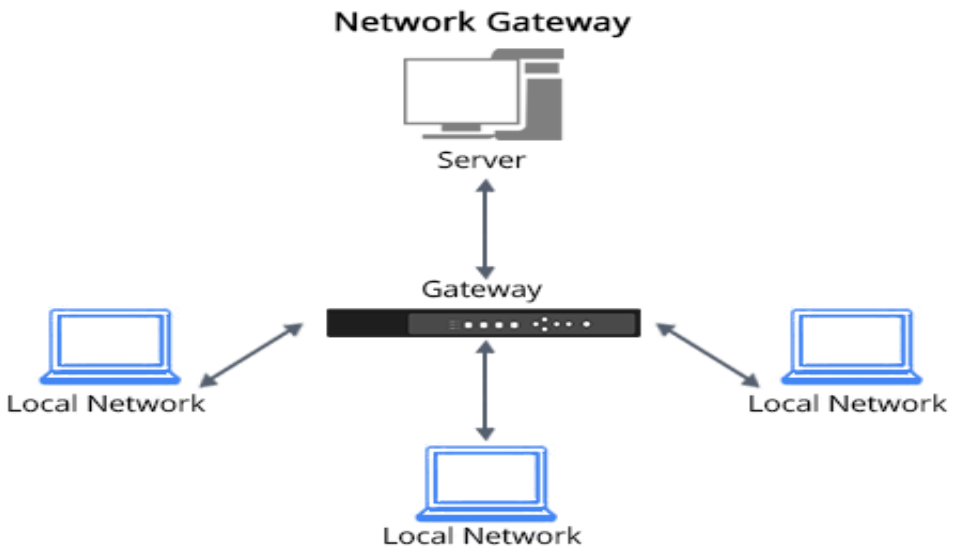
6. REPEATER: - A computer network is made up of various devices such as hubs, switches, modems, routers, and repeaters. Each of these devices plays a vital role in ensuring smooth communication between devices. Among them, the repeater is especially important for extending the reach of a network and maintaining the quality of data signals over long distances. A repeater is a network device that regenerates weakened or corrupted signals to restore them to their original form before retransmission. It operates at the physical layer (Layer 1) of the OSI model. Unlike an amplifier, which increases both signal and noise, a repeater reconstructs the original clean signal and forwards it, ensuring reliable data delivery over longer distances.

Repeaters are widely used in both wired networks (Ethernet, telephone lines, optical fiber) and wireless networks (Wi-Fi, radio, cellular communication).



7. GATEWAY: - A gateway is a network node or device that connects two networks that use different transmission protocols. Gateways play an important role in connecting two networks. It works as the entry-exit point for a network because all traffic that passes across the networks must pass through the gateway.

A gateway is a connecting point of any network that helps it to connect with different networks. The gateway monitors and controls all the incoming and outgoing traffic of the network. Suppose there are two different networks and they want to communicate with each other, so they need to set up a path between them. Now that path will be made between gateways of those different networks. Gateways are also known as protocol converters because they help to convert protocol supported by traffic of the different networks into that are supported by this network. Because of that, it makes smooth communication between two different networks.



CONCLUSION: - This experiment successfully provided a comparative study of fundamental networking devices: Hubs, Switches, and Routers, demonstrating their distinct roles, operational mechanisms.

EXPERIMENT NO: -2

Title: - (a) Construct Straight-Through and Cross-Over RJ-45 Ethernet cables.

(b) Construct a peer-to-peer connection using cross-over ethernet cable.

Part A: Construction of Straight-Through and Cross-Over RJ-45 Ethernet Cables

Objective: - To physically construct both a straight-through and a cross-over Ethernet cable according to the T568A and T568B wiring standards.

Required Materials: -

- Bulk Ethernet cable (CAT5e or CAT6)
- RJ-45 Connectors
- RJ-45 Crimping Tool (must include a stripper and cutter)
- Cable Tester (optional, but highly recommended for verification)

Theory: - RJ-45 connectors use eight pins. The T568A and T568B standards define the sequence of the eight internal colored wires to ensure proper transmission (Tx) and reception (Rx) of data signals. A straight-through cable uses the same standard on both ends to connect dissimilar devices (PC to Router), while a cross-over cable reverses the Tx and Rx lines by using T568A on one end and T568B on the other to connect similar devices (PC to PC).

Procedure: -

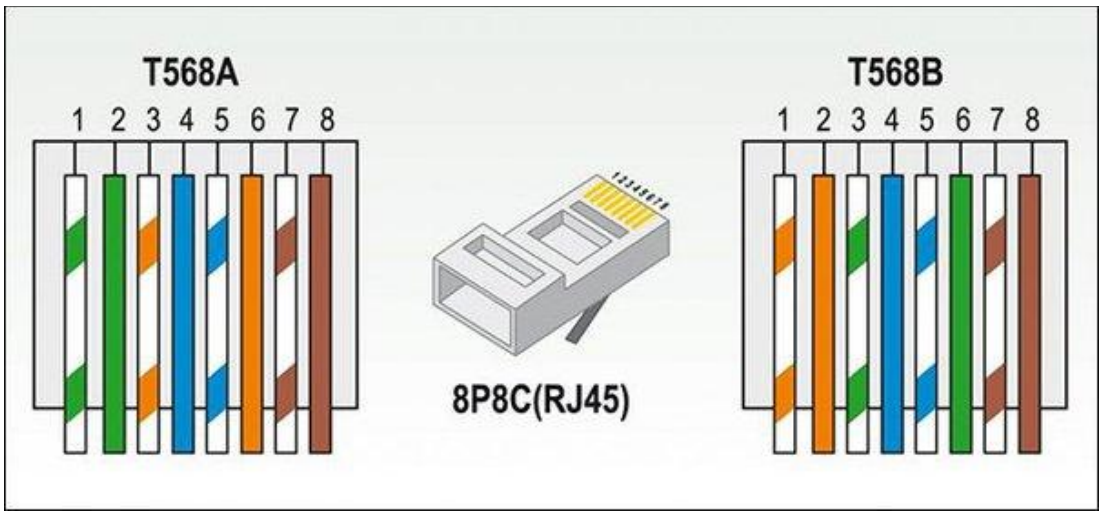
General Preparation: -

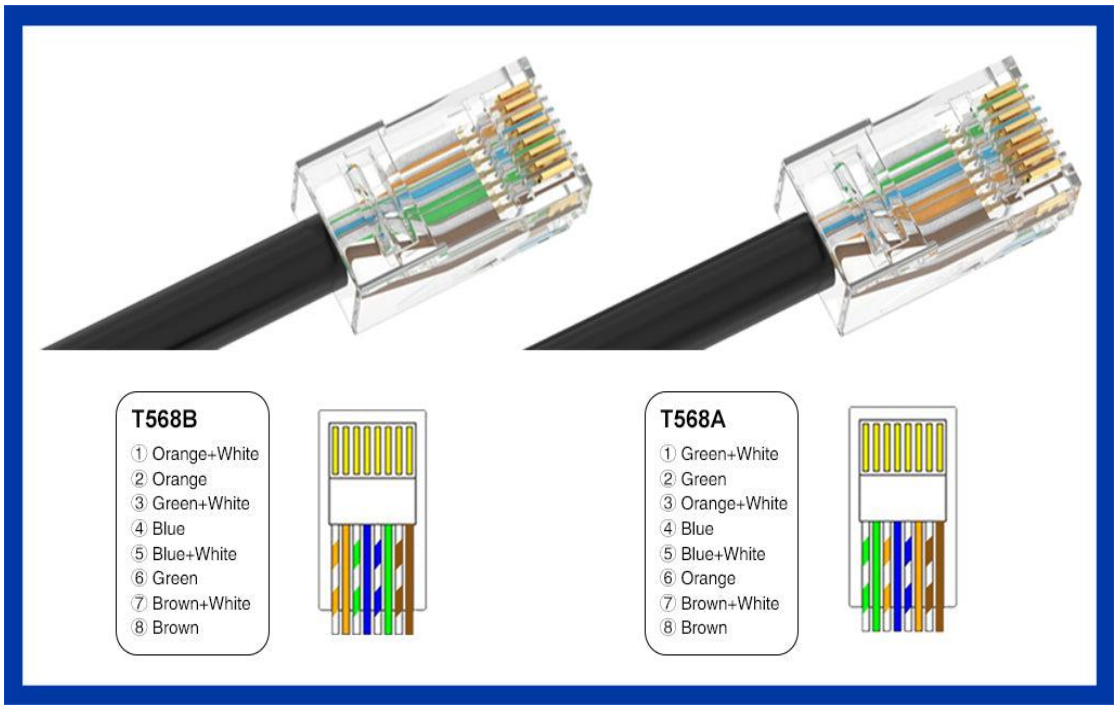
- **Strip the Outer Jacket:** Use the stripping blade on the crimping tool to carefully remove approximately 1 inch of the outer insulation jacket from one end of the Ethernet cable. Avoid nicking the internal wires.
- **Untwist and Arrange Wires:** Carefully untwist the four pairs of wires. Straighten them out to lay flat.

Wiring Sequence Standards: -

Pin	T568A Order	T568B Order
1	White/Green	White/Orange
2	Green	Orange
3	White/Orange	White/Green
4	Blue	Blue

5	White/Blue	White/Blue
6	Orange	Green
7	White/Brown	White/Brown
8	Brown	Brown





Constructing the Cable Ends: -

Arrange: Arrange the 8 wires in the required sequence (T568B is standard for most networks).

Trim: Ensure all wires are aligned perfectly straight and are the same length. Trim them neatly with the cutter on the crimping tool, leaving about 1/2 inch of exposed wire.

Insert: Hold the RJ-45 connector with the plastic retention clip facing downwards. Carefully insert the aligned wires into the connector pins, ensuring each wire slides into the correct channel and makes contact with the gold pins at the end. Verify the cable jacket seats within the connector body for strain relief.

Crimp: Insert the prepared connector into the RJ-45 slot of the crimping tool and squeeze firmly until the tool ratchets back to its open position. This action pushes the pins down to pierce the wires.

Repeat: Repeat steps on the second end of the cable, using the appropriate standard (T568B for straight-through, or T568A for crossover).

Verification: -

- Use a cable tester to confirm continuity and proper pin mapping for the designated cable type.

Part B: Constructing a Peer-to-Peer Connection Using a Cross-Over Cable

Objective: -

To establish direct network communication between two computers using the constructed cross-over Ethernet cable without the use of a network switch or router.

Required Materials: -

- Two (2) host computers (PC/Laptop) with Ethernet ports.
- One (1) functional Cross-Over Ethernet Cable (constructed in Part A).

Theory: -

A direct peer-to-peer connection requires manual configuration of network parameters because dynamic configuration protocols (DHCP) are unavailable. Both computers must be assigned a unique static IP address within the same subnet.

Procedure: -

Physical Connection: -

1. Plug one end of the cross-over cable into the Ethernet port of **Computer A**.
2. Plug the other end into the Ethernet port of **Computer B**.
3. Observe the link lights on both computers' Ethernet ports; they should illuminate, indicating a physical connection.

Network Configuration (Windows Example): -

1. Navigate to **Control Panel > Network and Sharing Center**.
2. Select **Change adapter settings** (or similar option to view network interfaces).
3. Right-click the **Ethernet** connection and select **Properties**.
4. Select **Internet Protocol Version 4 (TCP/IPv4)** and click **Properties**.
5. Select **Use the following IP address** and enter the following settings:

Setting	Computer A Values	Computer B Values
IP address:	`192.168.1.10`	`192.168.1.11`
Subnet mask:	`255.255.255.0`	`255.255.255.0`
Default gateway:	(Leave blank)	(Leave blank)

6. Click **OK** to save the settings on both computers.

Connection Verification (Ping Test): -

1. Open the Command Prompt (CMD) on Computer A.
2. Execute the ping command targeting Computer B's IP address:

```
ping 192.168.1.11
```

3. A successful connection is confirmed by receiving four replies with no packet loss.
4. Repeat the test from Computer B to Computer A (ping 192.168.1.10).

Results and Conclusion: -

The cross-over cable successfully enabled a direct physical link between the two host computers. Manual assignment of static IP addresses within the same subnet facilitated logical network communication, verified by successful ping tests. Peer-to-peer file sharing can now be configured and utilized between the hosts.

EXPERIMENT NO: -3

Title: - Study of network IP and set of Networking Commands.

Objective: - To study the concept of IP addressing and to understand the use of various basic networking commands in Windows/Linux for network configuration, testing, and troubleshooting.

Theory: -

IP Addressing: -

An IP address (Internet Protocol Address) is a unique numerical label assigned to each device connected to a computer network that uses the Internet Protocol.
Types of IP:

- IPv4: 32-bit address (e.g., 192.168.1.10)
- IPv6: 128-bit address (e.g., 2001:db8::1)

Components of an IP Address: -

- **Network ID:** Identifies the network segment.
- **Host ID:** Identifies the specific device in the network.
- **Subnet Mask:** Used to determine the network and host portions.
- **Default Gateway:** IP address of router used to connect outside local network.
- **DNS Server:** Resolves domain names to IP addresses.

Common Networking Commands: -
Below are the essential commands used in Windows and Linux:

(a) `ipconfig` / `ifconfig`

Displays IP configuration of the system.

Windows:

Ipconfig

Linux:

Ifconfig

(b) ping

Checks connectivity to a remote host.

Example:

ping google.com

(c) tracert / traceroute

Displays route taken by packets to reach destination.

Windows:

tracert google.com

Linux:

traceroute google.com

(d) nslookup

Checks DNS resolution for domain names.

nslookup www.example.com

(e) netstat

Displays active network connections and listening ports.

netstat -an

(f) arp

Displays or modifies ARP table.

arp -a

(g) route

Displays or modifies routing table.

route print (Windows)

route -n (Linux)

(h) hostname

Displays the system hostname.

Hostname

(i) ip (Linux)

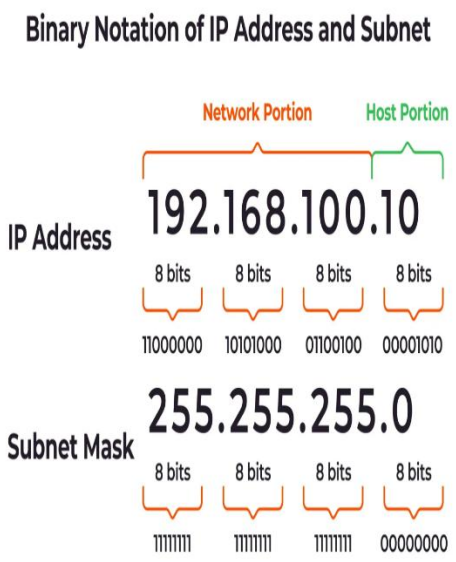
Advanced utility for network configuration.

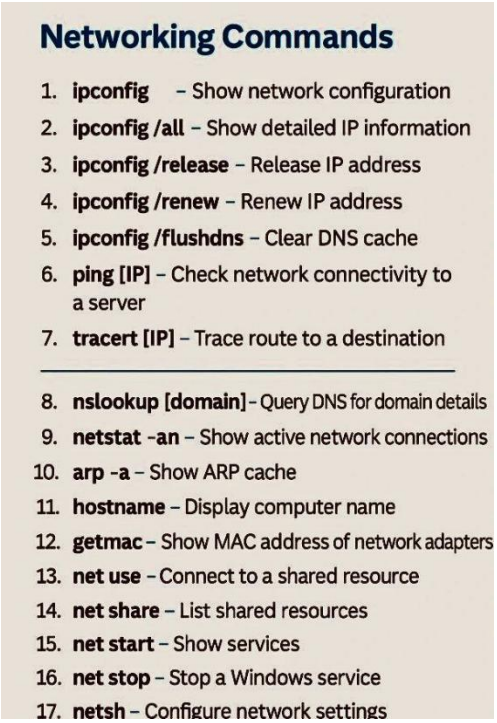
Examples:

ip a # Show IP details

ip r # Show routes

Diagram of IP Addressing and Networking Commands: -





Networking Commands

1. **ipconfig** – Show network configuration
2. **ipconfig /all** – Show detailed IP information
3. **ipconfig /release** – Release IP address
4. **ipconfig /renew** – Renew IP address
5. **ipconfig /flushdns** – Clear DNS cache
6. **ping [IP]** – Check network connectivity to a server
7. **tracert [IP]** – Trace route to a destination

---

8. **nslookup [domain]** – Query DNS for domain details
9. **netstat -an** – Show active network connections
10. **arp -a** – Show ARP cache
11. **hostname** – Display computer name
12. **getmac** – Show MAC address of network adapters
13. **net use** – Connect to a shared resource
14. **net share** – List shared resources
15. **net start** – Show services
16. **net stop** – Stop a Windows service
17. **netsh** – Configure network settings

REQUIREMENTS: -

- A computer with Windows/Linux OS
- Working network interface
- Internet connection (optional)

PROCEDURE: -

1. Open **Command Prompt** (Windows) or **Terminal** (Linux).
2. Type the command ipconfig (or ifconfig) and observe:
    - IP Address
    - Subnet Mask
    - Default Gateway
3. Use the ping command to check connectivity with local gateway and internet.
4. Run tracert / traceroute to analyze path to a known website.
5. Use nslookup to check the DNS response for a domain.
6. Execute netstat and note active connections.
7. Use arp -a to view MAC address mapping.
8. Record observations in the observation table.

OBSERVATION TABLE: -

Command	Output Details Observed
ipconfig / ifconfig	IP, subnet mask, gateway
Ping	Packet sent/received, loss %, time
tracert / traceroute	Hops, routers in path
Nslookup	DNS server, resolved IP
Netstat	Active connections, listening ports
arp -a	IP-to-MAC mapping
Route	Routing table details
ipconfig / ifconfig	IP, subnet mask, gateway

RESULT: - The IP addressing scheme of the system was studied, and various essential networking commands such as ipconfig, ping, netstat, tracert, nslookup, etc., were executed successfully. Each command's purpose and output were understood for network testing and troubleshooting.

CONCLUSION: - The experiment provided practical exposure to the fundamental networking commands used to check, verify, and troubleshoot network configuration and connectivity. Understanding IP addressing and commands enhances skills required for diagnosing network problems in both Windows and Linux environments.

EXPERIMENT NO: -4

Title: - Study of SSH and FTP server configuration.

Objective: - To study and configure SSH (Secure Shell) and FTP (File Transfer Protocol) servers for secure and non-secure remote access and file transfer operations in a networked environment.

Theory: -

1. SSH Server: - An SSH server provides secure remote login and command execution services over an encrypted connection.

It ensures confidentiality, authentication, and integrity.

Key Features: -

- Encrypted communication
- Secure login
- Remote command execution
- Secure tunneling
- File transfer (via SCP/SFTP)

Common SSH Server Software: -

- OpenSSH Server (Linux)
- OpenSSH for Windows
- Dropbear for embedded systems

2. FTP Server: -
An **FTP server** allows users to upload, download, and manage files over a network using the File Transfer Protocol.

Key Features: -

- File uploading and downloading
- Directory and file permission control
- User authentication
- Public or anonymous access option

Common FTP Server Software: -

- vsftpd (Linux)
- FileZilla FTP Server (Windows)
- ProFTPD (Linux)

REQUIREMENTS: -

- A computer with Windows/Linux OS
- Active network connection
- OpenSSH Server package
- vsftpd or FileZilla FTP Server
- Command prompt/terminal

PROCEDURE: -
A. SSH Server Configuration (Linux – OpenSSH Server)

1. **Install SSH Server:**

bash

sudo apt update

sudo apt install openssh-server

2. **Start and enable the SSH service:**

bash

sudo systemctl start ssh

sudo systemctl enable ssh

3. **Check service status:**

bash

sudo systemctl status ssh

4. **Find the system IP address:**

nginx

ifconfig

or

css

ip a

5. **Connect from another system:**

kotlin

ssh username@<server-ip>

6. **Verify successful login.**

B. FTP Server Configuration (Linux – vsftpd): -

1. Install FTP server:

bash

sudo apt install vsftpd

2. Start and enable the FTP service:

bash

sudo systemctl start vsftpd

sudo systemctl enable vsftpd

3. Edit configuration (optional):

bash

sudo nano /etc/vsftpd.conf

Common settings:

- Enable local users
- Disable anonymous login (optional)
- Allow uploads

4. Restart FTP service:

bash

sudo systemctl restart vsftpd

5. Test FTP connection from client system:

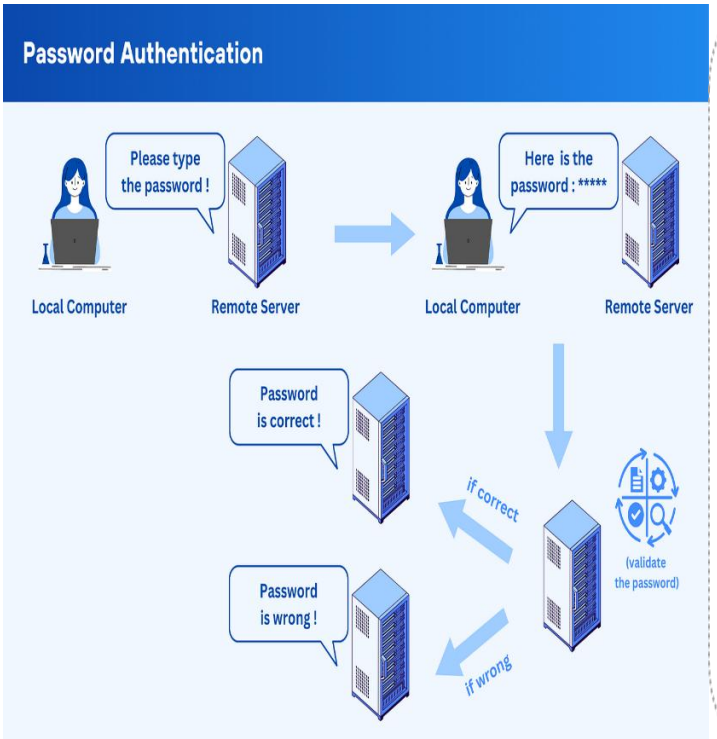
php-template

ftp <server-ip>

or use an FTP client like FileZilla.

6. Verify file upload, download, and directory access.

Diagram of SSH and FTP Server Configuration: -



OBSERVATION TABLE: -

Service	Command Executed	Observation
SSH	systemctl status ssh	SSH server running status
SSH	ssh user@server-ip	Successful remote login
FTP	systemctl status vsftpd	FTP server running status
FTP	ftp server-ip	Successful file transfer test

RESULT: -

The **SSH server** was successfully installed and configured to allow secure remote login. The **FTP server** was successfully configured to enable file uploading and downloading. Both services were tested and verified through client access.

CONCLUSION: -

This experiment helped us understand the configuration and working of SSH and FTP servers. SSH provides secure remote access through encrypted communication, while FTP offers an easy method of file transfer. The hands-on configuration strengthened practical knowledge of network administration and server management.

EXPERIMENT NO: -5

Title: - (a) Implement a peer-to-peer connection in CISCO Packet Tracer.

(b) Implement a LAN connection in CISCO Packet Tracer.

Objective: -

(a) To implement a **peer-to-peer connection** between two computers using Cisco Packet Tracer.

(b) To implement a **Local Area Network (LAN)** using a switch in Cisco Packet Tracer.

SOFTWARE REQUIRED: -

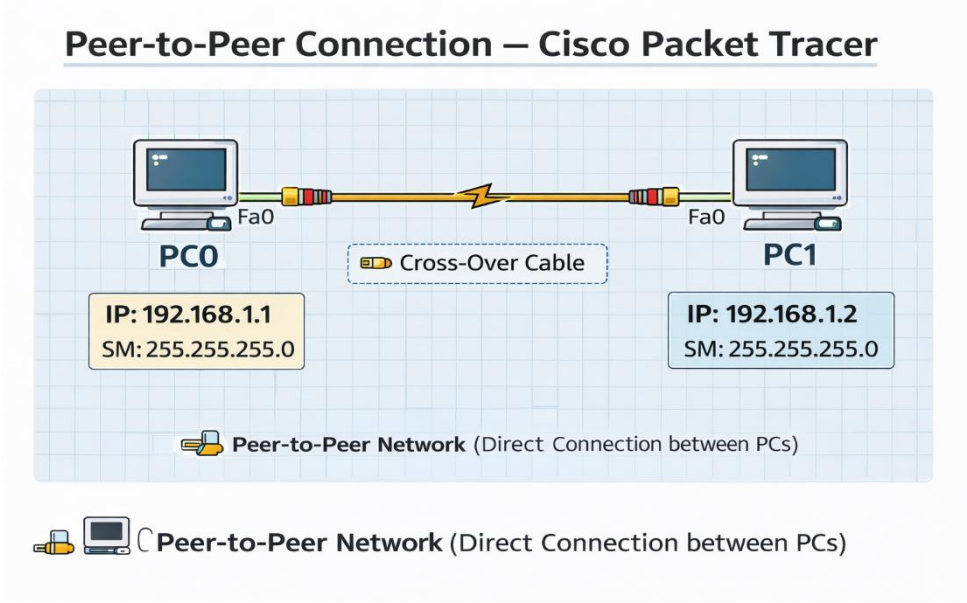
- Cisco Packet Tracer
- PC nodes
- Switch (for LAN)
- Ethernet cables

(a) Peer-to-Peer Connection

THEORY:

A **peer-to-peer (P2P) network** is a type of network where computers are directly connected to each other without using any intermediate networking device such as a switch or router. Each device can act as both client and server.

DIAGRAM:



PROCEDURE:

1. Open **Cisco Packet Tracer**.
2. Drag **two PCs** from the End Devices section.
3. Select **Copper Cross-Over cable**.
4. Connect **PC0 → PC1** using FastEthernet ports.
5. Configure IP address for both PCs:
   - **PC0:**
     IP Address: 192.168.1.1
     Subnet Mask: 255.255.255.0
   - **PC1:**
     IP Address: 192.168.1.2
     Subnet Mask: 255.255.255.0
6. Open **Command Prompt** on PC0.
7. Use the command:

ping 192.168.1.2

8. Observe the reply.

RESULT (a):

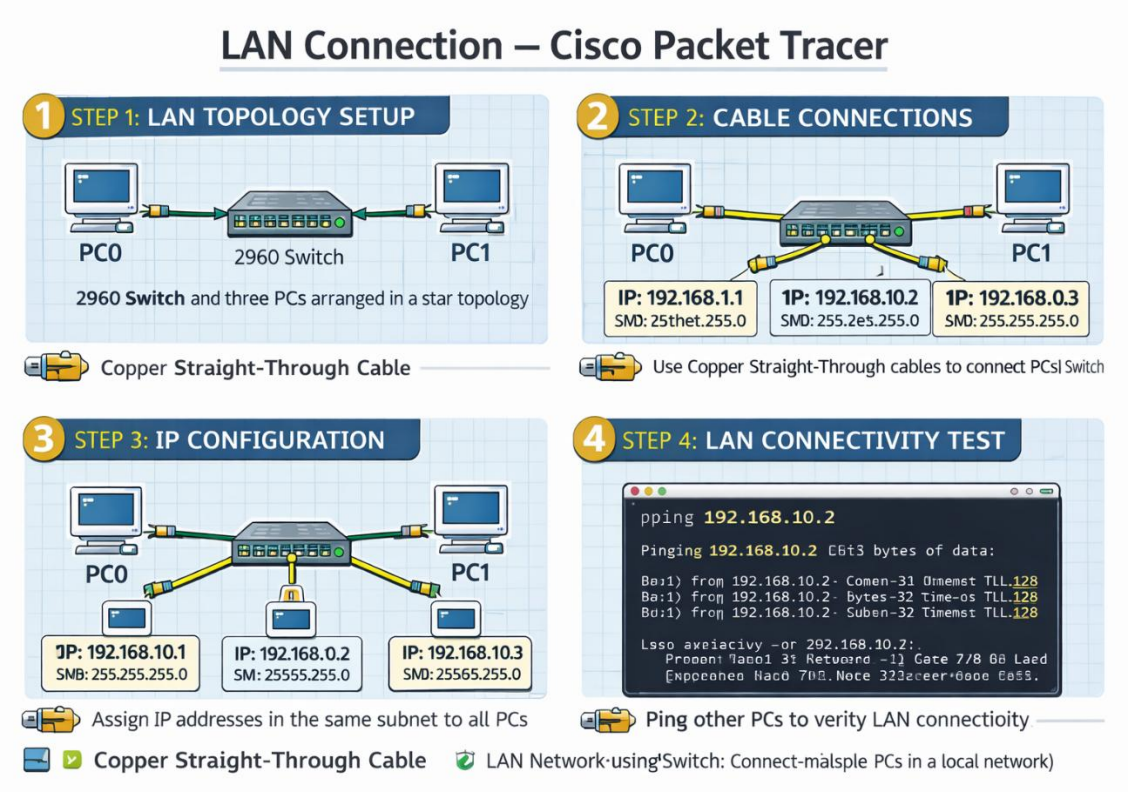
A peer-to-peer network was successfully implemented in Cisco Packet Tracer. Both computers were directly connected using a cross-over cable, and successful ping replies confirmed proper communication between the two systems.

(b) LAN Connection

THEORY:

A Local Area Network (LAN) connects multiple computers within a limited area such as a laboratory or office using a networking device like a switch. LAN allows resource sharing and efficient communication among connected devices.

DIAGRAM:



PROCEDURE:

1. Open Cisco Packet Tracer.
2. Drag one switch (2960) into the workspace.
3. Add three or more PCs.
4. Use Copper Straight-Through cables to connect:
   - PC0 → Switch
   - PC1 → Switch
   - PC2 → Switch
5. Assign IP addresses:
   - PC0: 192.168.10.1 / 255.255.255.0
   - PC1: 192.168.10.2 / 255.255.255.0
   - PC2: 192.168.10.3 / 255.255.255.0
6. Open Command Prompt on any PC.
7. Ping other PCs:

ping 192.168.10.2

ping 192.168.10.3

8. Observe packet transmission.

RESULT (b):

A LAN was successfully implemented in Cisco Packet Tracer using a switch. All connected PCs communicated effectively with each other, and successful ping responses verified proper LAN configuration.

CONCLUSION:

The experiments demonstrated the implementation of both peer-to-peer and LAN networks using Cisco Packet Tracer. The peer-to-peer network enabled direct communication between two systems, while the LAN allowed multiple systems to communicate efficiently through a switch. These experiments enhanced practical understanding of basic network topologies and IP configuration.

EXPERIMENT NO: - 6(a)

Title: - Configuration of a Router Using Static IP Addressing in Cisco Packet Tracer.

Aim

To configure a Cisco router with static IP addresses and enable communication between two different LAN networks using Cisco Packet Tracer.

Software & Hardware Requirements

- Computer System
- **Cisco Packet Tracer**
- 1 Router (Cisco 2911 or 1941)
- 2 Switches (Cisco 2960)
- 2 PCs
- **Copper Straight-through Cables**

Theory

A Router is a Layer 3 networking device that forwards data packets between different networks based on IP addresses. It maintains a routing table to determine the best path for packet delivery.

A Static IP address is manually configured on a device and does not change automatically. It is commonly used in:

- Servers
- Network devices (routers, switches)
- Small networks requiring fixed addressing

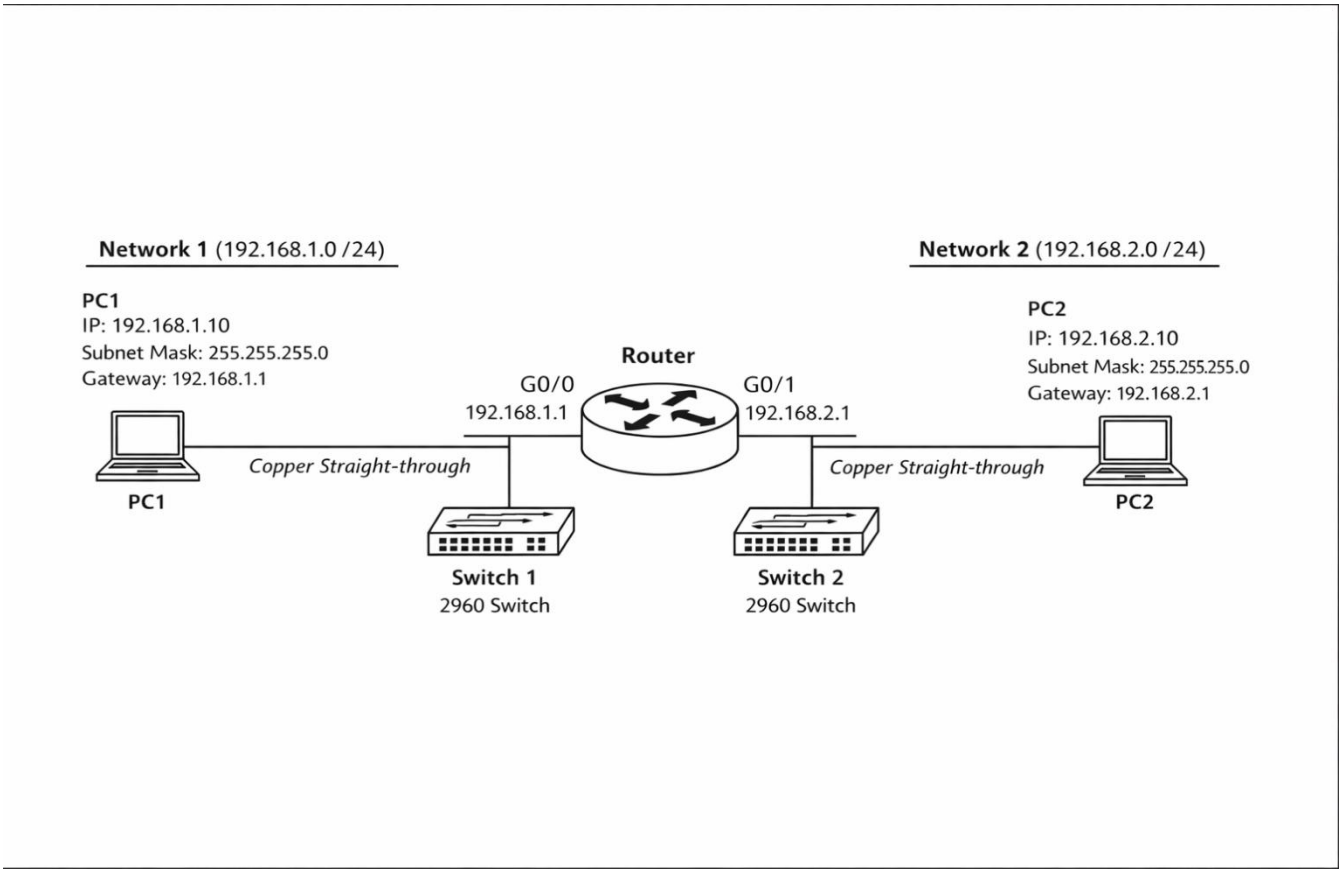
The Default Gateway is the IP address of the router interface that allows devices to communicate outside their local network.

Two different LANs are created:

- Network 1 → 192.168.1.0/24
- Network 2 → 192.168.2.0/24

Each LAN is connected to a separate router interface. The router routes traffic between the two networks.

Circuit Diagram: - Network topology with router and switches



IP Addressing Scheme

Device	Interface	IP Address	Subnet Mask	Default Gateway
Router	G0/0	192.168.1.1	255.255.255.0	N/A
Router	G0/1	192.168.2.1	255.255.255.0	N/A
PC1	FastEthernet	192.168.1.10	255.255.255.0	192.168.1.1
PC2	FastEthernet	192.168.2.10	255.255.255.0	192.168.2.1

Procedure

Step 1: Create the Topology

1. Open Cisco Packet Tracer.
2. From the bottom panel:
    - Drag and drop **Router 2911**.
    - Drag two **Switch 2960**.
    - Drag two **PCs**.
3. Connect devices using **Copper Straight-through cables**:
    - PC1 → Switch1
    - Switch1 → Router (G0/0)
    - Router (G0/1) → Switch2
    - Switch2 → PC2

Step 2: Configure Router Interfaces

1. Click on the Router.
2. Go to **CLI** tab.
3. Press Enter to continue.
4. Type the following commands:

```
enable
configure terminal

interface g0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit

interface g0/1
ip address 192.168.2.1 255.255.255.0
no shutdown
exit

exit
write memory
```

Explanation of Commands

- enable → Enters privileged mode
- configure terminal → Enters global configuration mode
- interface g0/0 → Selects interface
- ip address → Assigns IP address
- no shutdown → Activates interface
- write memory → Saves configuration

Step 3: Configure PC1

1. Click PC1.
2. Go to **Desktop → IP Configuration**.
3. Enter:

- IP Address: 192.168.1.10
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.1.1

Step 4: Configure PC2

1. Click PC2.
2. Go to **Desktop → IP Configuration**.
3. Enter:

- IP Address: 192.168.2.10
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.2.1

Verification

Check Router Configuration

On router CLI:

show ip interface brief

Interfaces should show **up/up** status.

Test Connectivity from PC1

Open PC1 → Desktop → Command Prompt:

ping 192.168.2.10

If configuration is correct, reply messages will appear.

Test Connectivity from PC2

ping 192.168.1.10

Successful replies confirm routing is working.

Observations

- Router interfaces changed from administratively down to up after using no shutdown.
- Both PCs successfully communicated.
- Static IP configuration enabled inter-network communication.

Result

The router was successfully configured with static IP addresses. Communication between two different networks was established and verified using ping command.

Common Errors & Troubleshooting

Problem	Solution
Request timed out	Check default gateway
Interface down	Use `no shutdown`
Wrong IP range	Verify IP addressing scheme
Cable issue	Check cable type

EXPERIMENT NO: -6(b)

Title: - Configure a Router on CISCO Packet Tracer using DHCP.

OBJECTIVES:

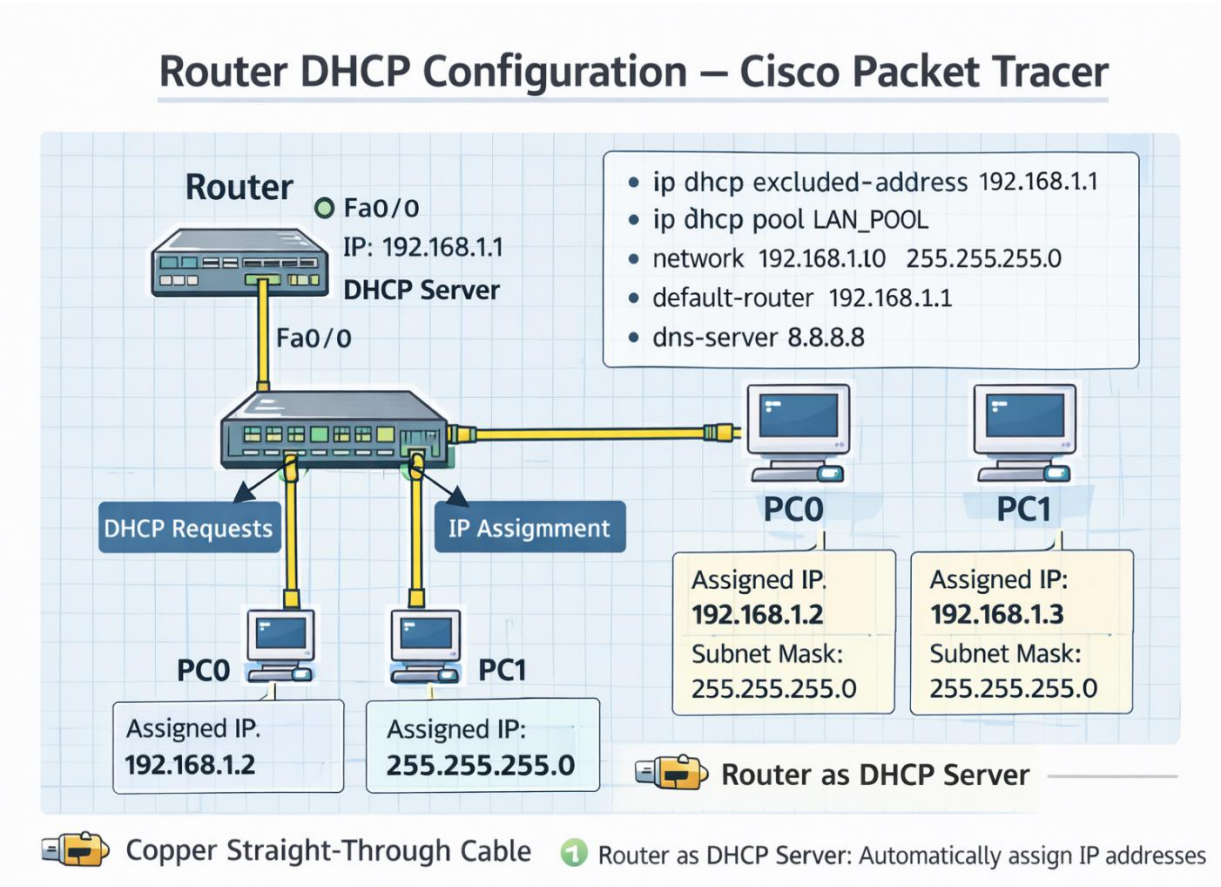
1. To understand the working of DHCP.
2. To configure a router to act as a DHCP server.
3. To verify automatic IP assignment to hosts.
4. To test network connectivity.

THEORY:

DHCP (Dynamic Host Configuration Protocol) is a network protocol used to automatically assign IP addresses and other network parameters such as subnet mask, default gateway, and DNS server to client devices.

When configured on a router, DHCP reduces manual configuration and minimizes IP address conflicts.

ROUTER-DHCP CONFIGURATION DIAGRAM:



REQUIREMENTS:

- Cisco Packet Tracer
- 1 Router (e.g., 2811)
- 1 Switch (2960)
- 2 or more PCs
- Copper Straight-Through Cables

NETWORK TOPOLOGY:

Router → Switch → PCs

(Star topology)

PROCEDURE:

Step 1: Create Network Topology

1. Open Cisco Packet Tracer.
2. Place one **Router**, one **Switch**, and two **PCs**.
3. Connect:
    - Router (Fa0/0) → Switch (Fa0/1)
    - Switch → PCs using straight-through cables.

Step 2: Configure Router Interface

1. Click on the router → **CLI**.
2. Enter the following commands:

```
Router> enable

Router# configure terminal

Router(config)# interface fastEthernet 0/0

Router(config-if)# ip address 192.168.1.1 255.255.255.0

Router(config-if)# no shutdown

Router(config-if)# exit
```

Step 3: Configure DHCP on Router

1. Exclude router IP from DHCP pool:

Router(config)# ip dhcp excluded-address 192.168.1.1

2. Create DHCP pool:

Router(config)# ip dhcp pool LAN_POOL

Router(dhcp-config)# network 192.168.1.0 255.255.255.0

Router(dhcp-config)# default-router 192.168.1.1

Router(dhcp-config)# dns-server 8.8.8.8

Router(dhcp-config)# exit

Step 4: Configure PCs

1. Open each PC → **Desktop → IP Configuration**.
2. Select **DHCP** option.
3. Observe that IP address, subnet mask, and gateway are assigned automatically.

Step 5: Verify Connectivity

1. Open Command Prompt on any PC.
2. Check IP address:

ipconfig

3. Ping router:

ping 192.168.1.1

4. Ping other PC to verify connectivity.

OBSERVATION TABLE:

Device	IP Address Assigned	Status
PC0	192.168.1.2	DHCP Successful
PC1	192.168.1.3	DHCP Successful
Router	192.168.1.1	Active

RESULT:

The router was successfully configured as a **DHCP server** in Cisco Packet Tracer. Client PCs received IP addresses automatically, and successful ping responses confirmed proper DHCP configuration and network connectivity.

CONCLUSION:

The experiment demonstrated the configuration and operation of DHCP on a router. Using DHCP simplifies network management by automatically assigning IP addresses and network parameters, ensuring efficient and error-free communication.

EXPERIMENT NO: - 6(c)

Title

Configuration of Dynamic Host Configuration Protocol (DHCP) using a Server in Cisco Packet Tracer

Aim

To configure a DHCP Server to automatically assign IP addresses to client PCs in a LAN using Cisco Packet Tracer.

Requirements

Hardware / Devices:

- 1 Router (2911 or 1941)
- 1 Switch (2960)
- 1 Server
- 2 PCs
- Copper Straight-through cables

Software:

- Cisco Packet Tracer

Theory

DHCP (Dynamic Host Configuration Protocol)

DHCP is a network protocol used to automatically assign IP addresses, subnet masks, default gateways, and DNS servers to client devices.

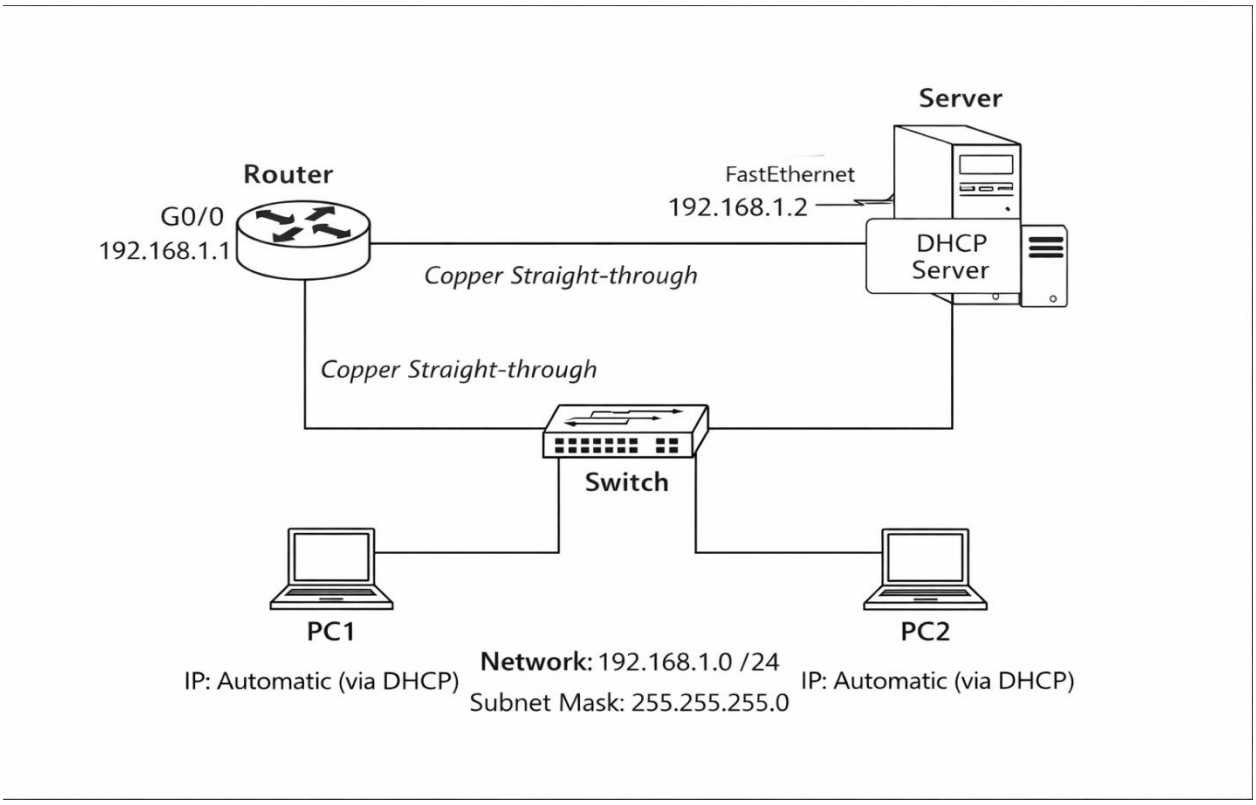
Why DHCP is Used

- Reduces manual configuration errors
- Saves time
- Centralized IP management
- Automatically avoids IP conflicts

DHCP Working Process (DORA Process)

1. **Discover** – Client sends broadcast message.
2. **Offer** – DHCP server offers IP address.
3. **Request** – Client requests offered IP.
4. **Acknowledge** – Server confirms assignment.

Circuit Diagram: **- DHCP network configuration diagram**



IP Addressing Scheme

Device	Interface	IP Address
Router	G0/0	192.168.1.1
Server	FastEthernet	192.168.1.2
PC1	DHCP	Automatic
PC2	DHCP	Automatic

Network: 192.168.1.0 /24
Subnet Mask: 255.255.255.0

Procedure

Step 1: Create Network Topology

1. Open Cisco Packet Tracer.
2. Add:
    - 1 Router
    - 1 Switch
    - 1 Server
    - 2 PCs
3. Connect all devices using Copper Straight-through cables:
    - PCs → Switch
    - Server → Switch
    - Router G0/0 → Switch

Step 2: Configure Router

1. Click Router → CLI
2. Enter:

```
enable
configure terminal
interface g0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit
end
write memory
```

Step 3: Configure Server with Static IP

1. Click Server.
2. Go to Desktop → IP Configuration.
3. Enter:
    - IP Address: 192.168.1.2
    - Subnet Mask: 255.255.255.0
    - Default Gateway: 192.168.1.1

Step 4: Configure DHCP Service on Server

1. Click Server.
2. Go to **Services** tab.
3. Click **DHCP**.
4. Turn DHCP Service **ON**.
5. Create a new Pool:

Fill the following details:

- Pool Name: LAN_POOL
- Default Gateway: 192.168.1.1
- DNS Server: 8.8.8.8
- Start IP Address: 192.168.1.10
- Subnet Mask: 255.255.255.0
- Maximum Users: 50

6. Click **Add**.

Step 5: Configure PCs to Obtain IP Automatically

For each PC:

1. Click PC → Desktop → IP Configuration.
2. Select **DHCP**.

The PC will automatically receive:

- IP Address
- Subnet Mask
- Default Gateway
- DNS Server

Verification

Check Assigned IP on PC

Go to PC → Command Prompt:

ipconfig

You should see an IP address like:

IP Address: 192.168.1.10
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.1.1

Test Connectivity

From PC1:

ping 192.168.1.1
ping 192.168.1.2 Successful replies confirm correct configuration.

Observations

- IP addresses were automatically assigned.
- No manual configuration required on PCs.
- DHCP server managed IP allocation efficiently.

Result

DHCP was successfully configured using a Server in Cisco Packet Tracer. The client PCs automatically obtained IP addresses and successfully communicated within the network.

Common Errors and Troubleshooting

Problem	Solution
PC not getting IP	Check DHCP service ON
Wrong gateway	Verify pool configuration
Server not reachable	Check static IP of server
Network not working	Verify cables and interface status

EXPERIMENT NO: -7

Title: - (a) Implementation of Cyclic Redundancy Check (CRC) for Error Detection using C.

(b) Implementation of Check-sum for Error detection using C.

OBJECTIVES:

1. To understand error detection mechanisms in data communication.
2. To implement CRC and Checksum algorithms using C language.
3. To detect transmission errors in data frames.
4. To compare the effectiveness of CRC and Checksum techniques.

THEORY:

(a) Cyclic Redundancy Check (CRC):

CRC is an error-detecting technique used to detect accidental changes to data during transmission. It treats the data as a binary number and performs polynomial division using a generator polynomial. The remainder obtained is appended to the data before transmission.

(b) Checksum:

Checksum is an error detection technique where data is divided into fixed-size segments, added together, and the complement of the sum is transmitted. At the receiver side, the sum is recomputed to check data integrity.

ALGORITHM:

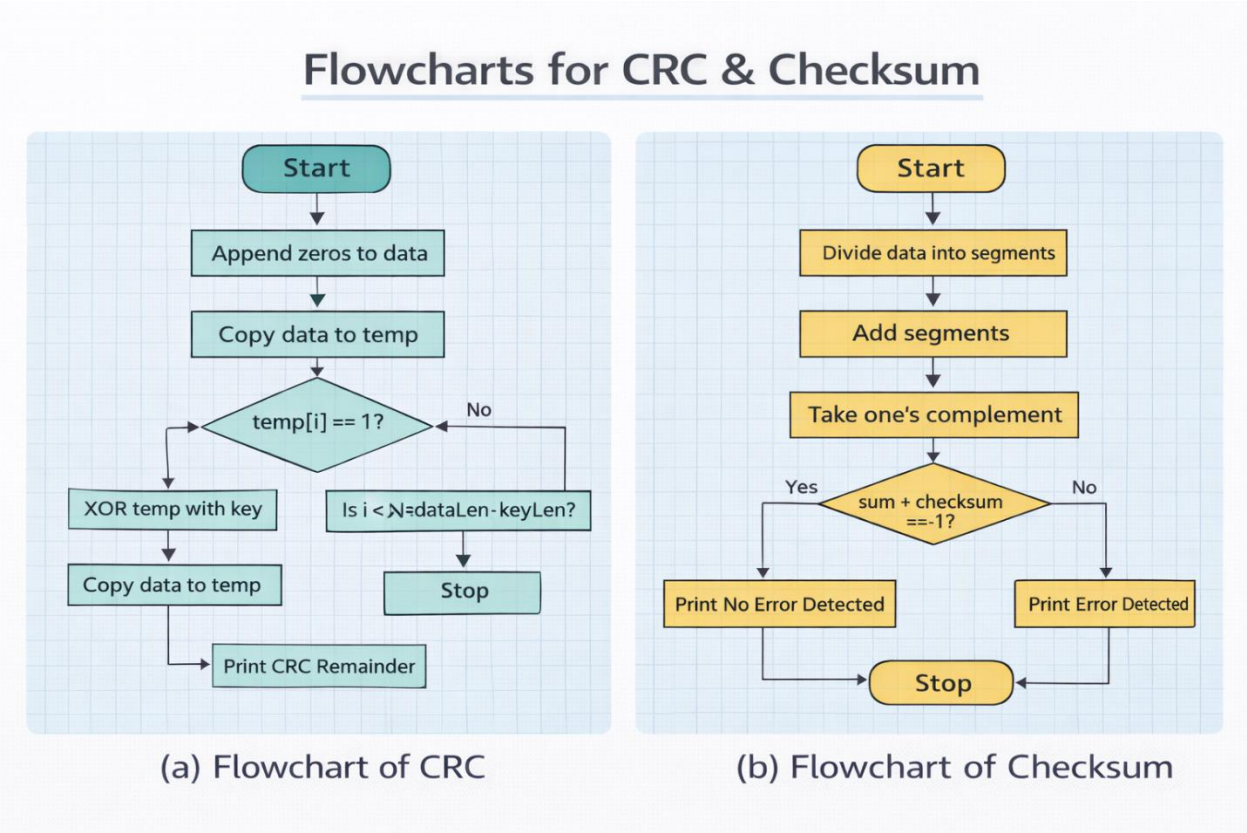
CRC Algorithm:

1. Append zeros to the data based on generator length.
2. Perform binary division using XOR operation.
3. Append the remainder to the original data.
4. At receiver side, divide received data again.
5. If remainder is zero → No error.

Checksum Algorithm:

1. Divide data into equal-sized segments.
2. Add all segments using binary addition.
3. Take one's complement of the sum.
4. Transmit checksum with data.
5. Receiver recalculates sum to detect error.

Flowcharts for CRC and Checksum: -



PROGRAM:

(a) CRC Program in C

```
#include <stdio.h>

#include <string.h>


void xorOperation(char *a, char *b, int len) {
```

```
    for (int i = 1; i < len; i++) {
        a[i] = (a[i] == b[i]) ? '0' : '1';
    }
}

void crc(char *data, char *key) {
    int dataLen = strlen(data);
    int keyLen = strlen(key);
    char temp[50];

    strcpy(temp, data);

    for (int i = 0; i <= dataLen - keyLen; i++) {
        if (temp[i] == '1') {
            xorOperation(temp + i, key, keyLen);
        }
    }

    printf("CRC Remainder: ");
    for (int i = dataLen - keyLen + 1; i < dataLen; i++) {
        printf("%c", temp[i]);
    }
}
```

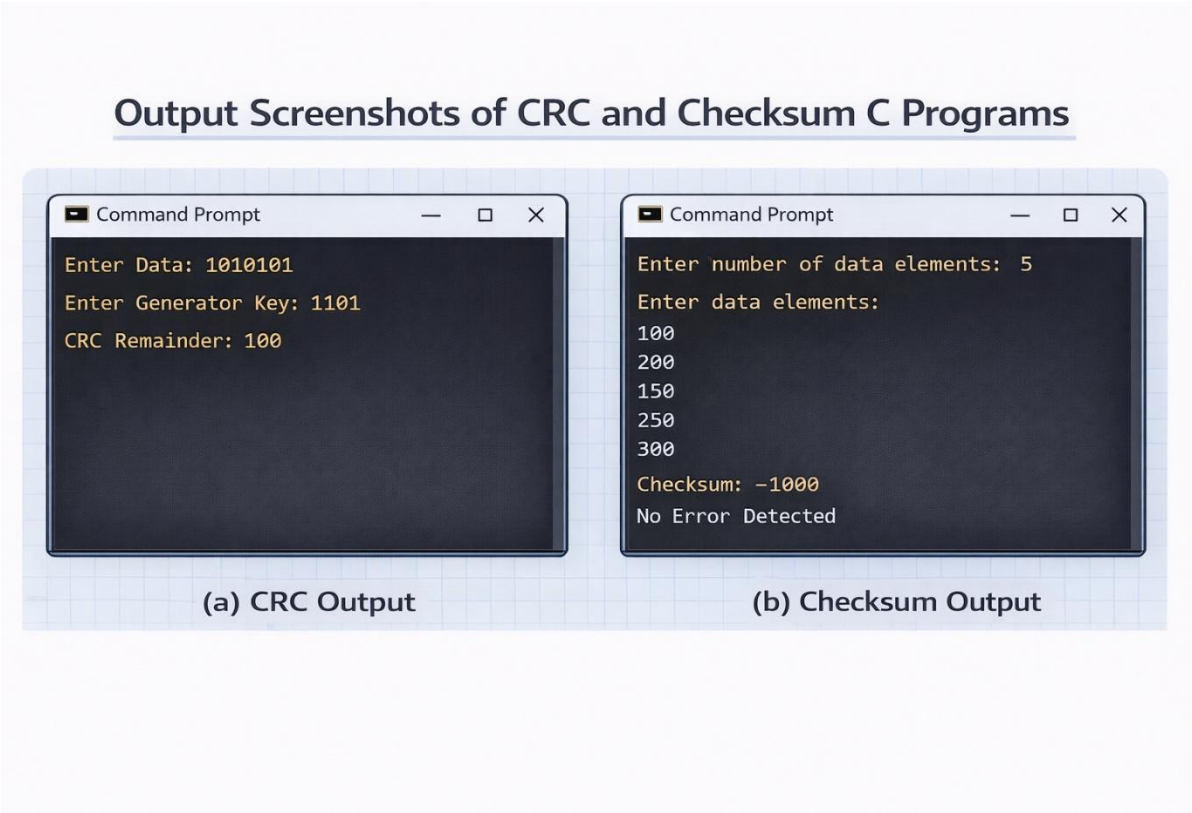
```
int main() {
    char data[50], key[50];

    printf("Enter Data: ");
    scanf("%s", data);
    printf("Enter Generator Key: ");
    scanf("%s", key);

    int keyLen = strlen(key);
    for (int i = 0; i < keyLen - 1; i++)
        strcat(data, "0");

    crc(data, key);
    return 0;
}
```

(b) Checksum Program in C

```
#include <stdio.h>

int main() {
    int n, sum = 0, data[50];

    printf("Enter number of data elements: ");
```

```
    scanf("%d", &n);

    printf("Enter data elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data[i]);
        sum += data[i];
    }

    int checksum = ~sum;

    printf("Checksum: %d\n", checksum);

    int receiverSum = sum + checksum;
    if (receiverSum == -1)
        printf("No Error Detected\n");
    else
        printf("Error Detected\n");

    return 0;
}
```

CRC and checksum program outputs: -



OBSERVATION:

- CRC accurately detected errors using polynomial division.
- Checksum detected errors using summation and complement method.
- CRC is more reliable for detecting burst errors.

RESULT:

(a) The **CRC algorithm** was successfully implemented using C and effectively detected transmission errors using generator polynomial division.
(b) The **Checksum algorithm** was successfully implemented using C and detected errors by verifying the complemented sum at the receiver side.

CONCLUSION:

Both CRC and Checksum techniques were implemented and tested successfully. CRC provides better error detection capability, especially for burst errors, whereas Checksum offers a simpler but less reliable method. These techniques play a crucial role in ensuring reliable data communication.

EXPERIMENT NO: -8

Title: - Implementation of client-server program with user input message with connection-less socket using C.

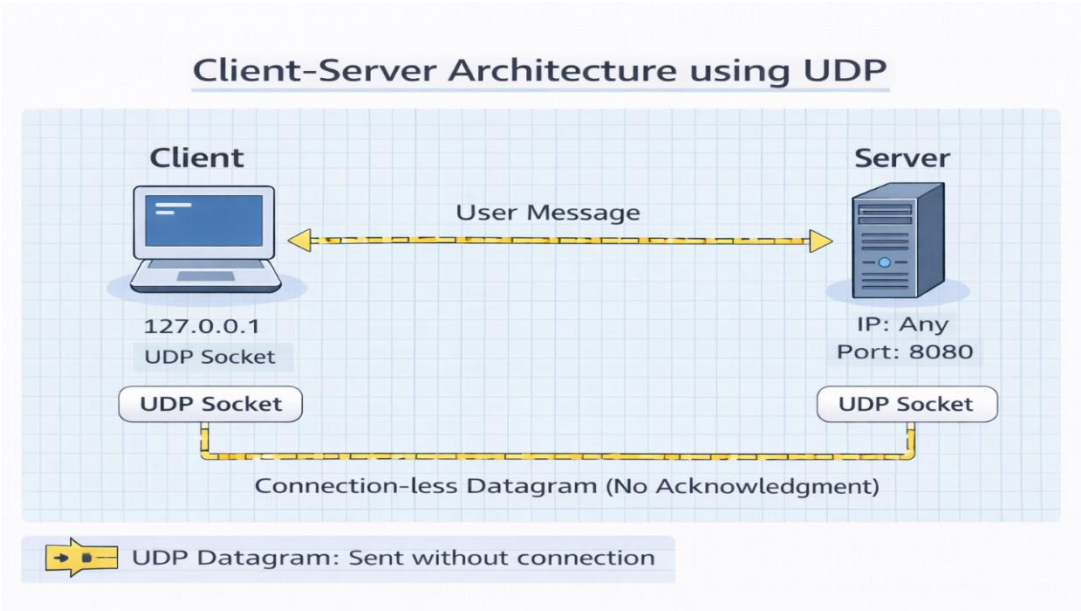
OBJECTIVES:

1. To understand connection-less communication using UDP.
2. To implement UDP client and server programs in C.
3. To exchange user-defined messages between client and server.
4. To observe message transmission without connection establishment.

THEORY:

A **connection-less socket** uses the **User Datagram Protocol (UDP)**, which does not establish a connection before data transfer. Each message (datagram) is sent independently, making UDP faster but less reliable than TCP.

Client-server architecture with UDP diagram: -



Features of UDP:

- No connection establishment
- Faster transmission
- No acknowledgment
- Suitable for real-time applications

REQUIREMENTS:

- C Compiler (GCC / Turbo C / Code Blocks)
- Linux or Windows OS
- Basic knowledge of socket programming

ALGORITHM:

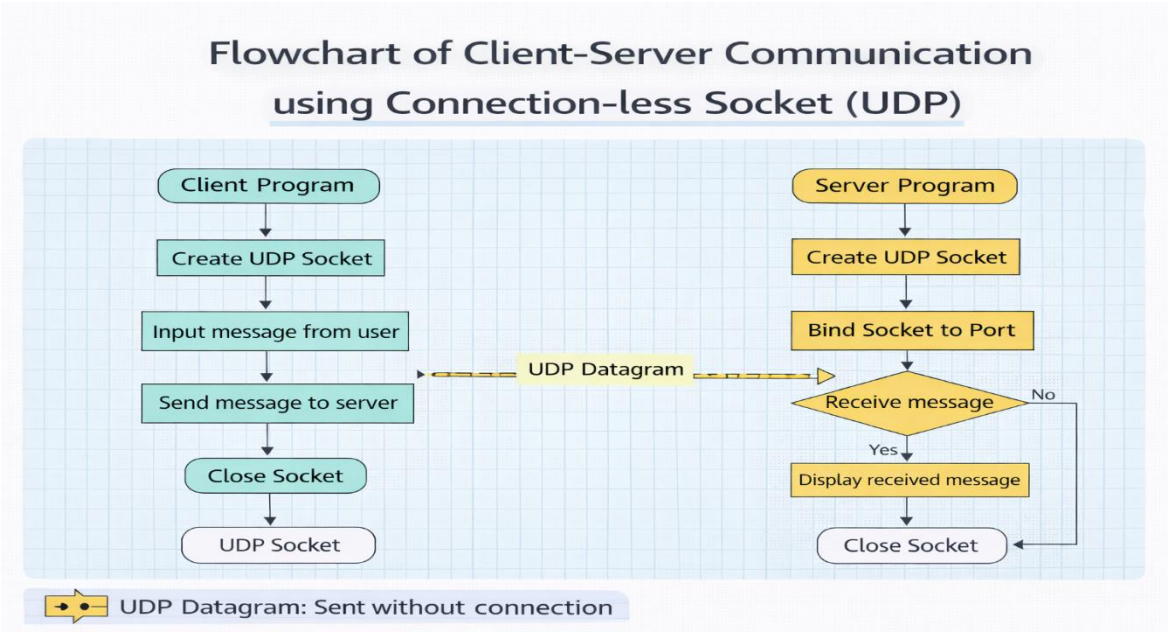
Server Side:

1. Create a UDP socket.
2. Bind socket to server IP and port.
3. Receive message from client.
4. Display received message.
5. Close socket.

Client Side:

1. Create a UDP socket.
2. Take message input from user.
3. Send message to server using sendto().
4. Close socket.

Client-server communication flowchart (UDP): -



PROGRAM:

UDP Server Program (server.c)

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t len = sizeof(clientAddr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    bind(sockfd, (struct sockaddr*)&serverAddr, sizeof(serverAddr));

    recvfrom(sockfd, buffer, sizeof(buffer), 0,
            (struct sockaddr*)&clientAddr, &len);

    printf("Message from Client: %s\n", buffer);

    close(sockfd);
    return 0;
}
```

UDP Client Program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in serverAddr;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    printf("Enter message to send: ");
    fgets(buffer, sizeof(buffer), stdin);

    sendto(sockfd, buffer, strlen(buffer), 0,
           (struct sockaddr*)&serverAddr, sizeof(serverAddr));

    close(sockfd);
    return 0;
}
```

PROCEDURE:

1. Open terminal/command prompt.
2. Compile server program:

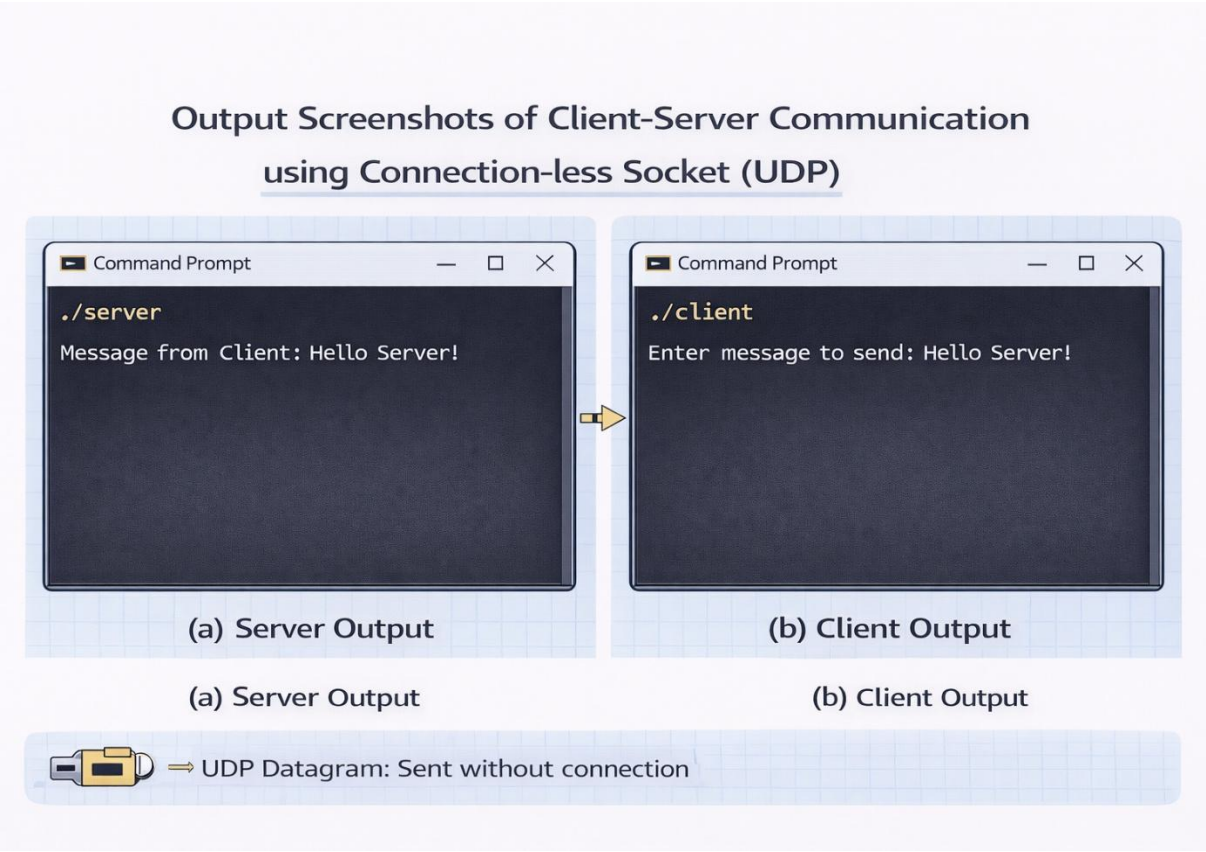
   gcc server.c -o server
3. Compile client program:

   gcc client.c -o client
4. Run server first:

   ./server
5. Run client in another terminal:

   ./client
6. Enter message at client side.
7. Observe message received at server.

Client-server communication via UDP: -



OBSERVATION:

Component	Observation
Socket Type	UDP (Connection-less)
Data Transfer	Message sent without connection
Reliability	No acknowledgment
Output	Message received correctly

RESULT:

The client-server program using connection-less socket (UDP) was successfully implemented in C. The client sent a user-defined message, and the server received and displayed it without establishing a connection.

CONCLUSION:

This experiment demonstrated the working of UDP-based client-server communication. Since UDP does not require connection establishment, it provides faster data transmission but without reliability guarantees, making it suitable for real-time applications.

EXPERIMENT NO: -9

Title: - Implementation of the client-server program with connection-oriented socket using C

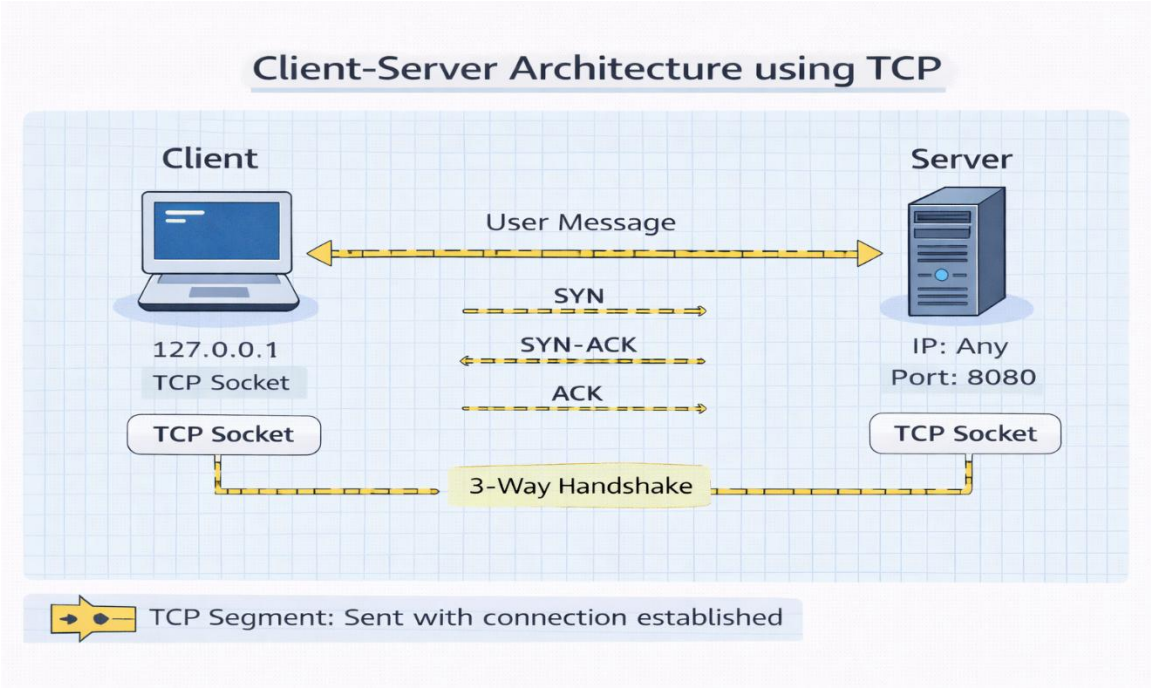
OBJECTIVES:

1. To understand connection-oriented communication using TCP.
2. To implement TCP client and server programs in C.
3. To establish a reliable connection between client and server.
4. To exchange data using socket programming functions.

THEORY:

A **connection-oriented socket** uses the **Transmission Control Protocol (TCP)**, which establishes a reliable connection before data transfer. TCP guarantees **ordered, error-free, and reliable** delivery of data between client and server.

TCP Client-Server architecture diagram: -



Features of TCP:

- Connection establishment (3-way handshake)
- Reliable and ordered data delivery
- Error detection and recovery
- Flow control

REQUIREMENTS:

- C Compiler (GCC / Turbo C / CodeBlocks)
- Linux or Windows OS
- Basic knowledge of socket programming

ALGORITHM:

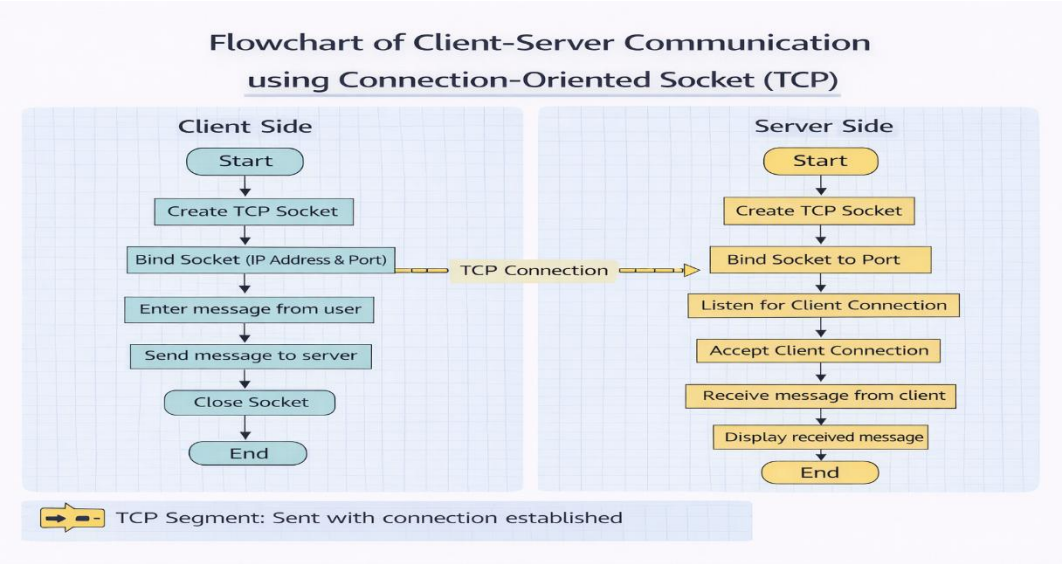
Server Side:

1. Create a TCP socket.
2. Bind the socket to an IP address and port number.
3. Listen for incoming client connections.
4. Accept client connection.
5. Receive message from client.
6. Display received message.
7. Close the socket.

Client Side:

1. Create a TCP socket.
2. Connect to the server.
3. Accept message input from user.
4. Send message to server.
5. Close the socket.

TCP client-server communication flowchart: -



PROGRAM:

TCP Server Program (server.c)

```
#include <stdio.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>


int main() {

    int serverSocket, clientSocket;

    char buffer[1024];

    struct sockaddr_in serverAddr, clientAddr;

    socklen_t addrSize;


    serverSocket = socket(AF_INET, SOCK_STREAM, 0);


    serverAddr.sin_family = AF_INET;

    serverAddr.sin_port = htons(8080);

    serverAddr.sin_addr.s_addr = INADDR_ANY;


    bind(serverSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr));


    listen(serverSocket, 5);
```

```
    printf("Server is waiting for connection...\n");

    addrSize = sizeof(clientAddr);
    clientSocket = accept(serverSocket, (struct sockaddr*)&clientAddr, &addrSize);

    recv(clientSocket, buffer, sizeof(buffer), 0);
    printf("Message from Client: %s\n", buffer);

    close(clientSocket);
    close(serverSocket);

    return 0;
}
```

TCP Client Program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int clientSocket;
    char buffer[1024];
```

```
    struct sockaddr_in serverAddr;

    clientSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(clientSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr));

    printf("Enter message to send: ");
    fgets(buffer, sizeof(buffer), stdin);

    send(clientSocket, buffer, strlen(buffer), 0);

    close(clientSocket);

    return 0;
}
```

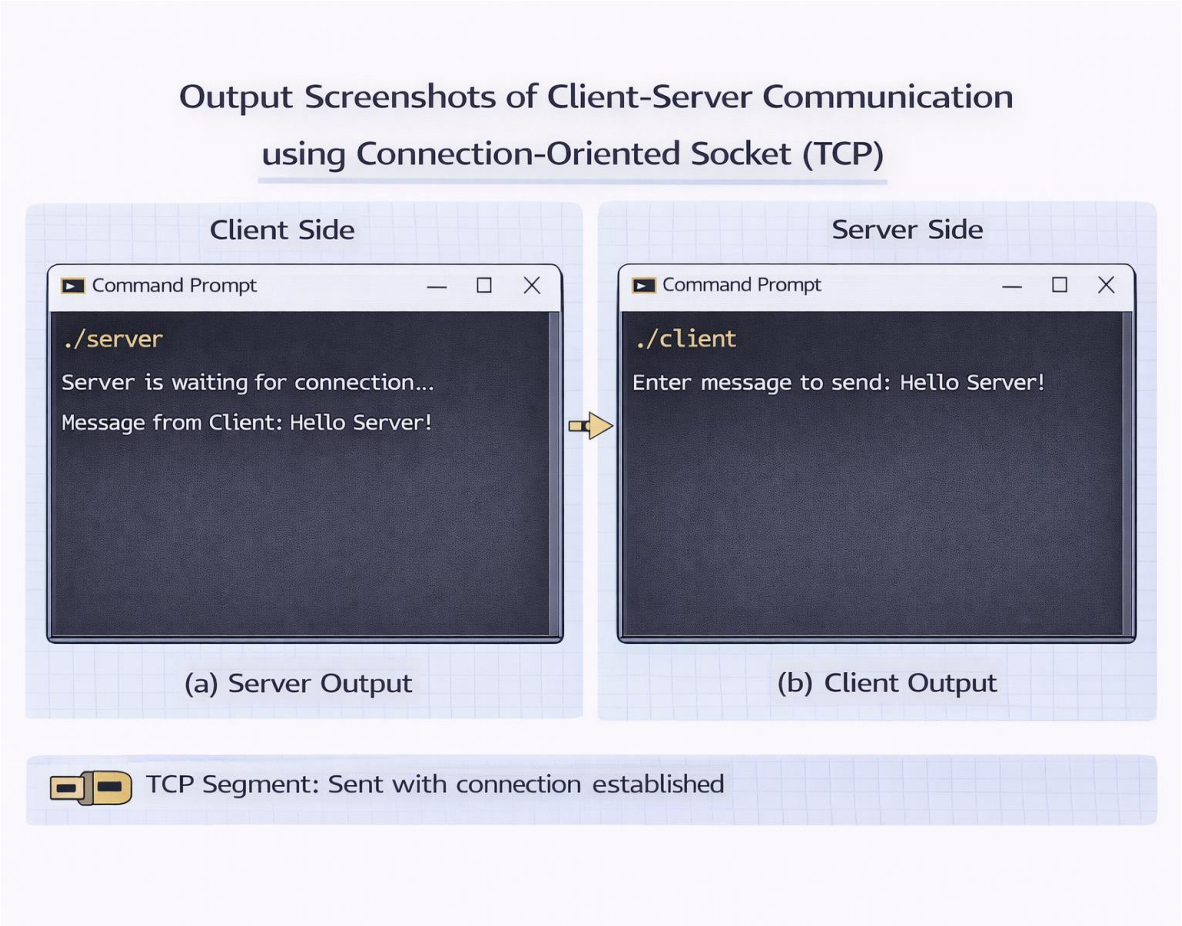
PROCEDURE:

1. Open terminal/command prompt.
2. Compile server program:

   gcc server.c -o server

3. Compile client program:
   gcc client.c -o client
4. Run the server first:
   ./server
5. Run the client in another terminal:
   ./client
6. Enter message at client side.
7. Observe received message at server side.

TCP client-server communication in action: -



OBSERVATION:

Parameter	Observation
Protocol Used	TCP
Socket Type	Connection-oriented
Reliability	High
Data Transfer	Ordered and error-free
Output	Message successfully received

RESULT:

The client-server program using **connection-oriented sockets (TCP)** was successfully implemented in C. The client established a connection with the server and transmitted a user-defined message reliably.

CONCLUSION:

This experiment demonstrated TCP-based client-server communication. Since TCP ensures reliable and ordered data transfer, it is widely used in applications like web browsing, file transfer, and email services.

EXPERIMENT NO: -10

Title: - Implementation of the multi-client and single Chat server program for the connection-oriented socket programming using C.

OBJECTIVES:

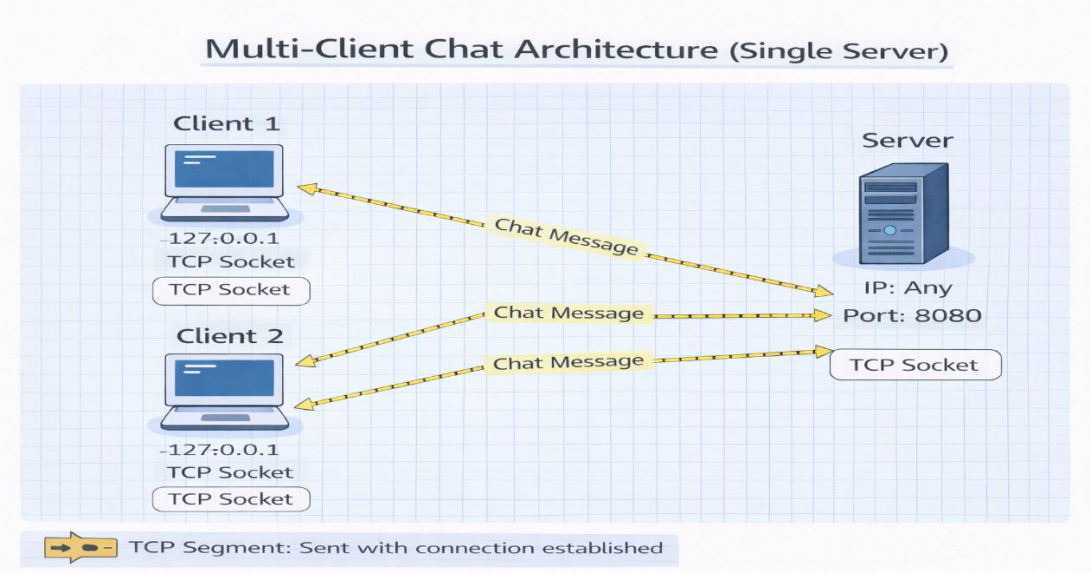
1. To understand multi-client communication using TCP.
2. To implement a single server handling multiple clients.
3. To use fork() / multiple processes for concurrent client handling.
4. To exchange chat messages between server and clients reliably.

THEORY:

In **connection-oriented socket programming**, TCP establishes a reliable connection before data transfer.

A **multi-client chat server** allows multiple clients to connect to one server simultaneously. The server handles each client independently, commonly using **process creation (fork())** or **threading**.

Multi-client chat architecture diagram: -



Key Characteristics:

- Reliable data transfer
- One server, multiple clients
- Uses TCP sockets
- Each client handled concurrently

REQUIREMENTS:

- C Compiler (GCC / CodeBlocks)
- Linux OS (preferred for fork())
- Terminal / Command Prompt

ALGORITHM:

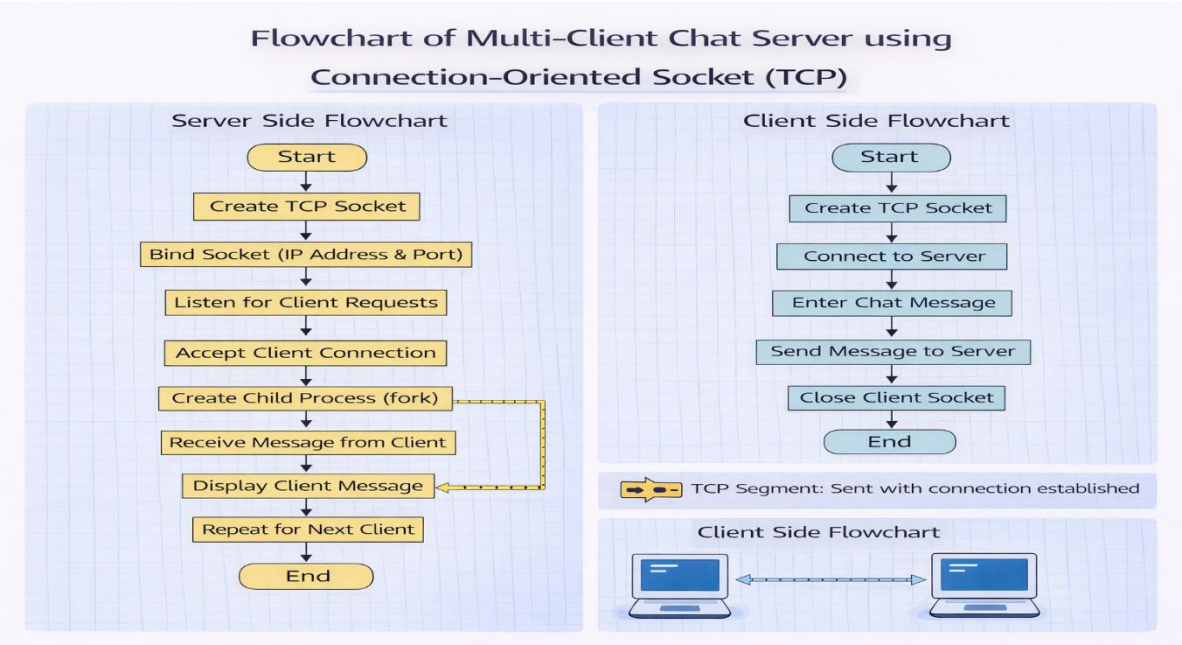
Server Side:

1. Create TCP socket.
2. Bind socket to IP address and port.
3. Listen for incoming client connections.
4. Accept client connection.
5. Create child process for each client using fork().
6. Receive and display messages from client.
7. Close client socket after communication.

Client Side:

1. Create TCP socket.
2. Connect to server.
3. Enter chat message from user.
4. Send message to server.
5. Close socket.

Multi-client chat server flowchart: -



PROGRAM:

Multi-Client Chat Server Program (server.c)

```
#include <stdio.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>


int main() {

    int serverSocket, clientSocket;

    struct sockaddr_in serverAddr, clientAddr;

    char buffer[1024];

    socklen_t addrSize;


    serverSocket = socket(AF_INET, SOCK_STREAM, 0);


    serverAddr.sin_family = AF_INET;

    serverAddr.sin_port = htons(8080);

    serverAddr.sin_addr.s_addr = INADDR_ANY;


    bind(serverSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr));


    listen(serverSocket, 5);

    printf("Chat Server started...\n");
```

```
    while (1) {
        addrSize = sizeof(clientAddr);
        clientSocket = accept(serverSocket,
                              (struct sockaddr*)&clientAddr,
                              &addrSize);

        if (fork() == 0) {
            recv(clientSocket, buffer, sizeof(buffer), 0);
            printf("Client Message: %s\n", buffer);
            close(clientSocket);
            return 0;
        }
        close(clientSocket);
    }

    close(serverSocket);
    return 0;
}
```

Chat Client Program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
```

```
#include <arpa/inet.h>

int main() {
    int clientSocket;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    clientSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(clientSocket,
            (struct sockaddr*)&serverAddr,
            sizeof(serverAddr));

    printf("Enter message: ");
    fgets(buffer, sizeof(buffer), stdin);

    send(clientSocket, buffer, strlen(buffer), 0);
    close(clientSocket);
```

```
    return 0;
}
```

PROCEDURE:

1. Open terminal.
2. Compile server program:

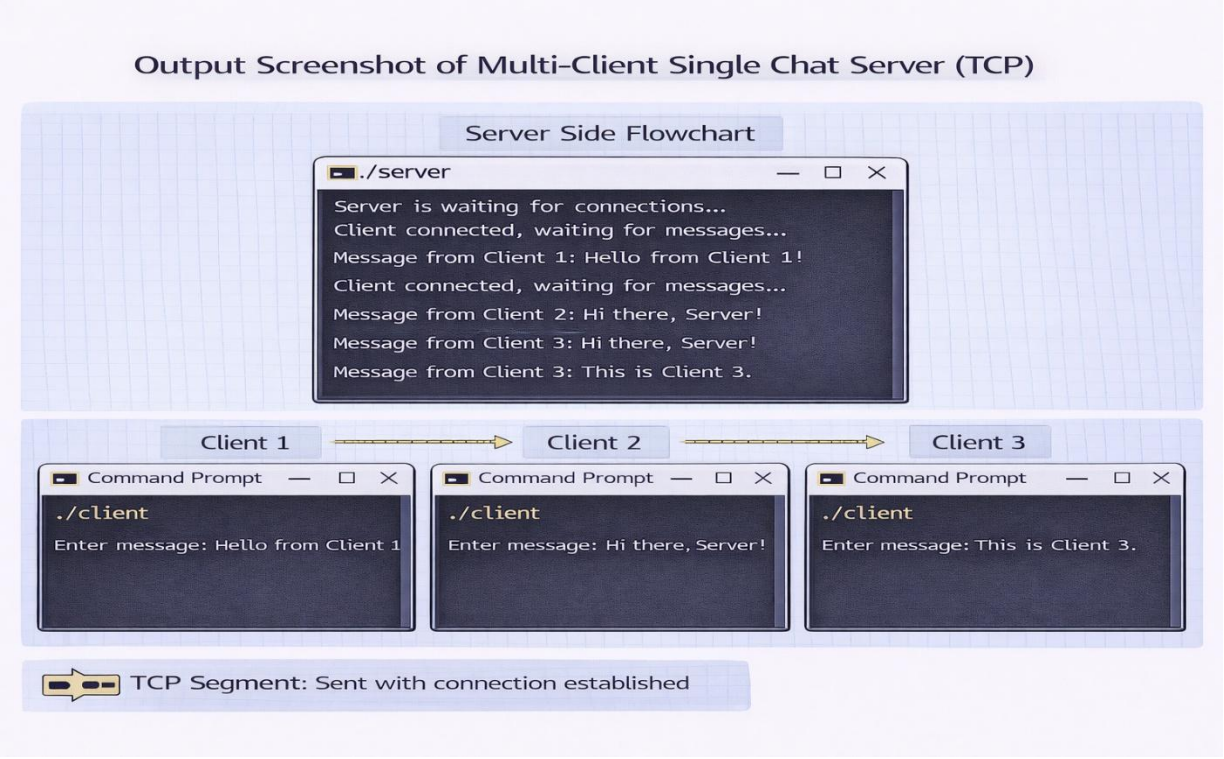
   gcc server.c -o server
3. Compile client program:

   gcc client.c -o client
4. Run server:

   ./server
5. Open multiple terminals for clients.
6. Run each client:

   ./client
7. Enter messages from different clients.
8. Observe messages displayed on server.

Multi-client chat server setup: -



OBSERVATION TABLE:

Parameter	Observation
Socket Type	TCP
Server Type	Single server
Client Type	Multiple clients
Communication	Reliable
Concurrency	Supported using fork()

RESULT:

The **multi-client single chat server** was successfully implemented using **connection-oriented socket programming (TCP)** in C. Multiple clients were able to connect to the server and send messages reliably.

CONCLUSION:

This experiment demonstrated the implementation of a TCP-based multi-client chat server. By using process creation, the server efficiently handled multiple client connections simultaneously, ensuring reliable communication.